220 Curated MCQs | CUET PG 2026/2027 | Full Course Coverage | 100% Solved

CUET PG Computer Science & MCA 200+ Master Question Bank & Solutions

220 Unit-Wise High-Yield MCQs | 4 Options | Step-by-Step Hinglish Solutions | Difficulty Levels | Common Exam Traps

Key Features & Course Overview

- 220 High-Yield MCQs covering the complete CUET PG / MCA Data Structures & Algorithms syllabus.
- Unit 1 to 12 systematic coverage: Basics, Arrays, Stacks, Queues, Linked Lists, Recursion, Trees, BST/AVL, B-Trees, Hashing, Graphs, Sorting & Searching.
- Every question includes 4 clear options, exact answer key, and difficulty rating (Easy, Medium, Difficult).
- In-depth step-by-step Hinglish explanations showing WHY the correct option is right and WHY alternatives fail.
- Special "Exam Trap & Shortcut" notes for tricky numericals, boundary conditions, and NTA patterns.
- Structured with Questions first (Section A) and Solutions immediately following (Section B) for active practice.

Important Preparation Directive:

Attempt all 220 questions under timed conditions (approx. 1.2 to 1.5 minutes per question). Review every incorrect attempt with the detailed Hinglish solution.

Question Bank Index - 12 Units (220 High-Yield MCQs)

Yeh master question bank CUET PG Computer Science / MCA syllabus ke har topic se 220 most expected, conceptual aur numerical questions ko cover karta hai. Har unit mein questions pehle attempt karne ke liye Section A mein diye gaye hain, aur unke theek baad Section B mein step-by-step Hinglish solutions aur exam traps diye gaye hain:

Unit 1: Introduction to Data Structures & Algorithm Analysis (Q1 - Q18) [18 Questions]
Unit 2: Arrays & Memory Address Calculations (Q19 - Q36) [18 Questions]
Unit 3: Stacks & Polish Notation Expressions (Q37 - Q54) [18 Questions]
Unit 4: Queues & Circular Queues (Q55 - Q72) [18 Questions]
Unit 5: Linked Lists (Q73 - Q90) [18 Questions]
Unit 6: Recursion & Recurrences (Q91 - Q105) [15 Questions]
Unit 7: Trees & Binary Tree Properties (Q106 - Q125) [20 Questions]
Unit 8: Binary Search Trees & AVL Trees (Q126 - Q145) [20 Questions]
Unit 9: B-Trees & B+ Trees (Q146 - Q163) [18 Questions]
Unit 10: Hashing & Collision Resolution (Q164 - Q182) [19 Questions]
Unit 11: Graph Theory & Traversals (Q183 - Q202) [20 Questions]
Unit 12: Searching & Sorting Algorithms (Q203 - Q220) [18 Questions]

[INFO] Instructions for Solving & Score Maximization

- Total Questions in this Master Bank: 220 MCQs.
- Active Recall Strategy: Pehle questions ko bina solutions dekhe rough paper par attempt karein.
- Target Speed: 1.2 to 1.5 minutes per question (CUET PG Exam benchmark).
- Detailed Solutions: Har question ke baad Hinglish explanation se samjhein ki galat options kyu galat hain.
- Common Traps: Special "Exam Trap" notes ko dhyan se padhein kyunki NTA/CUET PG aksar boundary conditions par questions frame karta hai.

Unit 1: Introduction to Data Structures & Algorithm Analysis (Q1 - Q18)

SECTION A: PRACTICE QUESTIONS

Q1. [Easy] Which of the following is a non-linear data structure?

A) Stack
B) Queue
C) Tree
D) Array

Q2. [Easy] An Abstract Data Type (ADT) defines:

A) How data is stored in hardware
B) What operations are performed, but not how they are implemented
C) Only the memory allocation algorithm
D) The exact C++ syntax for the data structure

Q3. [Medium] Which asymptotic notation represents the strict upper bound of a function?

A) Big-O (O)
B) Little-o (o)
C) Big-Omega (Omega)
D) Big-Theta (Theta)

Q4. [Medium] Arrange the following functions in increasing order of growth rate:
f1(n) = 2^n, f2(n) = n^(3/2), f3(n) = n log n, f4(n) = n!

A) f3 < f2 < f1 < f4
B) f2 < f3 < f1 < f4
C) f3 < f2 < f4 < f1
D) f2 < f3 < f4 < f1

Q5. [Easy] What is the time complexity of the following code snippet?

```
int count = 0;
for (int i = 1; i <= n; i *= 2)
   count++;
```

A) O(n)
B) O(n^2)
C) O(log n)
D) O(sqrt(n))

Q6. [Easy] Visiting each element in a data structure exactly once is known as:

A) Sorting
B) Traversing
C) Merging
D) Searching

Q7. [Easy] Which of the following is a static data structure?

A) Array
B) Singly Linked List
C) Dynamic Stack
D) Binary Search Tree

Q8. [Medium] If an algorithm takes 0.5 milliseconds for input size n = 100, and its complexity is O(n^2), how long will it take for n = 200?

A) 1.0 ms
B) 2.0 ms
C) 4.0 ms
D) 8.0 ms

Q9. [Easy] The best case time complexity of an algorithm is mathematically represented by:

A) O (Big-O)
B) Omega (Big-Omega)
C) Theta (Big-Theta)
D) o (Little-o)

Q10. [Easy] Which data structure is best suited to represent a hierarchical relationship among data items?

A) Array
B) Stack
C) Tree
D) Queue

Q11. [Medium] What is the time complexity of the following nested loop?

```
for (int i = 0; i < n; i++)
   for (int j = 0; j < i; j++)
      sum++;
```

A) O(n)
B) O(n log n)
C) O(n^2)
D) O(n^3)

Q12. [Medium] Which of the following statements about Big-Theta (Theta) notation is TRUE?

A) f(n) = Theta(g(n)) iff f(n) = O(g(n)) and f(n) = Omega(g(n))
B) Big-Theta represents only worst case
C) Big-Theta represents only best case
D) Big-Theta is always linear

Q13. [Easy] What is the complexity of an algorithm that performs 3n^3 + 100n^2 + 5000n operations?

A) O(n)
B) O(n^2)
C) O(n^3)
D) O(n!)

Q14. [Easy] Which data structure allows both homogeneous and heterogeneous data representation depending on implementation?

A) Array (Homogeneous) vs Structure/Record (Heterogeneous)
B) Stack vs Queue
C) Tree vs Graph
D) Singly vs Doubly list

Q15. [Medium] **What is the time complexity of the following code?**

```
for (int i = 1; i <= n; i++)
   for (int j = 1; j <= n; j *= 2)
      k++;
```

A) O(n)
B) O(n log n)
C) O(n^2)
D) O(log n)

Q16. [Easy] **If f(n) = 100n and g(n) = n^2, which of the following is correct?**

A) f(n) = O(g(n))
B) g(n) = O(f(n))
C) f(n) = Theta(g(n))
D) f(n) = Omega(g(n))

Q17. [Easy] **Space complexity of an algorithm considers:**

A) Only the input space
B) Fixed part (instruction space) + Variable part (dynamic memory & recursion stack)
C) Only the hard disk space
D) Only global variables

Q18. [Medium] **Which asymptotic notation is symmetric (i.e., if f(n) is related to g(n), then g(n) is related to f(n))?**

A) Big-O
B) Big-Omega
C) Big-Theta (Theta)
D) Little-o

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 1)

QID	Correct Option	Difficulty	Concept Tested
Q1	Option C	Easy	Which of the following is a non-linear dat...
Q2	Option B	Easy	An Abstract Data Type (ADT) defines:...
Q3	Option B	Medium	Which asymptotic notation represents the s...
Q4	Option A	Medium	Arrange the following functions in increas...
Q5	Option C	Easy	What is the time complexity of the followi...
Q6	Option B	Easy	Visiting each element in a data structure ...
Q7	Option A	Easy	Which of the following is a static data st...
Q8	Option B	Medium	If an algorithm takes 0.5 milliseconds for...
Q9	Option B	Easy	The best case time complexity of an algori...
Q10	Option C	Easy	Which data structure is best suited to rep...
Q11	Option C	Medium	What is the time complexity of the followi...
Q12	Option A	Medium	Which of the following statements about Bi...
Q13	Option C	Easy	What is the complexity of an algorithm tha...
Q14	Option A	Easy	Which data structure allows both homogeneo...
Q15	Option B	Medium	What is the time complexity of the followi...
Q16	Option A	Easy	If f(n) = 100n and g(n) = n^2, which of th...
Q17	Option B	Easy	Space complexity of an algorithm considers...
Q18	Option C	Medium	Which asymptotic notation is symmetric (i....

[INFO] Solution for Q1: Correct Option [C]

- Explanation: Tree aur Graph non-linear data structures hote hain kyunki inme data elements sequential order mein nahi balki hierarchical ya interconnected network relation mein hote hain. Stack, Queue, aur Array linear structures hain.
- Exam Trap & Shortcut: Kayi students Linked List ko non-linear samajh lete hain kyunki usme pointers hote hain. Yaad rakhein Linked List linear hai!

[INFO] Solution for Q2: Correct Option [B]

- Explanation: ADT ek mathematical model hota hai jo sirf yeh specify karta hai ki data structure par "KYA" operations kiye ja sakte hain (jaise PUSH, POP), lekin yeh nahi batata ki implementation array se hoga ya linked list se.
- Exam Trap & Shortcut: ADT implementation-independent hota hai.

[INFO] Solution for Q3: Correct Option [B]

- Explanation: Big-O represents upper bound (<=), jabki Little-o represents STRICT upper bound (<). Yani Little-o mein function kabhi bhi bound ke asymptotically equal nahi ho sakta.
- Exam Trap & Shortcut: Big-O equality allow karta hai (f(n) = O(f(n))), lekin Little-o strict inequality chahta hai.

[INFO] Solution for Q4: Correct Option [A]

- Explanation: Growth rates: n log n (f3) < n^1.5 (f2) < 2^n (exponential, f1) < n! (factorial, f4). Isliye sahi sequence hai f3 < f2 < f1 < f4.
- Exam Trap & Shortcut: Always remember: Exponential (2^n) grows much slower than Factorial (n!).

[INFO] Solution for Q5: Correct Option [C]

- Explanation: Loop variable i har iteration mein 2 se multiply ho raha hai (1, 2, 4, 8, ..., 2^k). Jab 2^k >= n hoga, loop terminate ho jayega. Therefore, k = log2(n), so time complexity is O(log n).
- Exam Trap & Shortcut: Agar i++ hota toh O(n) hota, lekin yahan i *= 2 hai jo logarithmic hota hai.

[INFO] Solution for Q6: Correct Option [B]

- Explanation: Traversing ka matlab hota hai data structure ke har element ko exactly ek baar access/process karna.
- Exam Trap & Shortcut: Searching ka matlab kisi specific key ki location find karna hota hai, har element ko visit karna nahi.

[INFO] Solution for Q7: Correct Option [A]

- Explanation: Array ek static data structure hai kyunki iska size compile time par fix ho jata hai aur runtime par change nahi ho sakta.
- Exam Trap & Shortcut: Linked lists, trees aur dynamically allocated stacks runtime par grow karte hain.

[INFO] Solution for Q8: Correct Option [B]

- Explanation: Time T(n) = c * n^2. Jab input size 100 se 200 hota hai (2 times double), toh time (2)^2 = 4 times badh jata hai. Therefore: 0.5 ms * 4 = 2.0 ms.
- Exam Trap & Shortcut: Students often double the time (0.5 * 2 = 1.0) forgetting the quadratic n^2 square factor!

[INFO] Solution for Q9: Correct Option [B]

- Explanation: Big-Omega (Omega) lower bound ko denote karta hai, jo algorithm ke best-case time ko mathematically represent karta hai.
- Exam Trap & Shortcut: Big-O upper bound (worst case) hota hai.

[INFO] Solution for Q10: Correct Option [C]

- Explanation: Tree ek hierarchical data structure hai (jaise organization chart, family tree, folder system). Graphs network relationship represent karte hain.
- Exam Trap & Shortcut: Graph is for general network relationships with cycles; Tree is strictly hierarchical and acyclic.

[INFO] Solution for Q11: Correct Option [C]

- Explanation: Outer loop runs n times. Inner loop runs 0 + 1 + 2 + ... + (n-1) times = n(n-1)/2 times. Expanding gives (n^2 - n)/2 = O(n^2).
- Exam Trap & Shortcut: Inner loop variable j depends on i; sum of first n integers is quadratic O(n^2).

[INFO] Solution for Q12: Correct Option [A]

- Explanation: Formal Theorem: f(n) = Theta(g(n)) agar aur sirf agar f(n) dono O(g(n)) [upper bound] aur Omega(g(n)) [lower bound] satisfy karta ho. Isse Tight Bound kehte hain.
- Exam Trap & Shortcut: Theta is asymptotically tight, sandwiching the function between c1*g(n) and c2*g(n).

[INFO] Solution for Q13: Correct Option [C]

- Explanation: Asymptotic analysis mein hum highest degree term (dominant term) ko rakhte hain aur constants drop karte hain. Yahan dominant term 3n^3 hai, so complexity is O(n^3).
- Exam Trap & Shortcut: Low order terms like 100n^2 and large constant coefficients become negligible as n approaches infinity.

[INFO] Solution for Q14: Correct Option [A]

- Explanation: Array strictly homogeneous (same data type) hota hai. Struct/Class heterogeneous (different data types) elements ko group karta hai.
- Exam Trap & Shortcut: Arrays in C/C++ can never store mixed data types.

[INFO] Solution for Q15: Correct Option [B]

- Explanation: Outer loop runs n times. Inner loop doubles j every iteration, running log2(n) times. Total executions = n * log2(n) = O(n log n).
- Exam Trap & Shortcut: Do not multiply to get O(n^2); the inner loop is logarithmic, not linear.

[INFO] Solution for Q16: Correct Option [A]

- Explanation: Linear function 100n quadratic function n^2 se asymptotically bohot slow grow karta hai. For n > 100, 100n <= 1 * n^2. Therefore, f(n) = O(g(n)).
- Exam Trap & Shortcut: The coefficient 100 does not matter for large n.

[INFO] Solution for Q17: Correct Option [B]

- Explanation: Total Space = Fixed space (code size, constants) + Variable space (dynamically allocated memory, recursion stack, local variables).
- Exam Trap & Shortcut: Recursion stack memory is an essential part of auxiliary space complexity.

[INFO] Solution for Q18: Correct Option [C]

- Explanation: Big-Theta is symmetric: f(n) = Theta(g(n)) if and only if g(n) = Theta(f(n)). Big-O and Big-Omega are transpose-symmetric, not symmetric.
- Exam Trap & Shortcut: f(n) = O(g(n)) implies g(n) = Omega(f(n)), NOT g(n) = O(f(n)).

Unit 2: Arrays & Memory Address Calculations (Q19 - Q36)

SECTION A: PRACTICE QUESTIONS

Q19. [Medium] An array A[1..10, 1..15] is stored in Row-Major Order. If the base address is 1000 and each element occupies 2 bytes, what is the address of A[4][6]?

A) 1102
B) 1100
C) 1114
D) 1110

Q20. [Medium] Given a 2D array A[1..10, 1..15] stored in Column-Major Order with base address 1000 and 2 bytes per element, what is the address of element A[4][6]?

A) 1106
B) 1100
C) 1104
D) 1108

Q21. [Easy] What is the number of elements in a 2D array defined as A[-3..5, 2..8]?

A) 54
B) 63
C) 48
D) 56

Q22. [Easy] Which programming language stores 2D arrays in Column-Major Order by default?

A) C
B) C++
C) FORTRAN
D) Java

Q23. [Easy] What is the time complexity to insert an element at the beginning of an array of size n?

A) O(1)
B) O(log n)
C) O(n)
D) O(n^2)

Q24. [Easy] Why does an array provide O(1) random access to its elements?

A) Because elements are sorted
B) Because elements are stored in contiguous memory and have uniform size
C) Because it uses pointers
D) Because of hash functions

Q25. [Medium] A triangular matrix of size n x n has non-zero elements only on and below the main diagonal. How many elements need to be stored?

A) n^2
B) n(n - 1) / 2
C) n(n + 1) / 2
D) 2n - 1

Q26. [Easy] **If an array is declared as `int arr[5] = {10, 20, 30};` in C, what is the value of `arr[4]`?**

A) Garbage value
B) 0
C) 30
D) Compilation error

Q27. [Difficult] **What is the time complexity of deleting an element from index i in an unsorted array if order does NOT need to be preserved?**

A) O(n)
B) O(log n)
C) O(1)
D) O(n log n)

Q28. [Easy] **Given base address B = 500, element size W = 4, LB = -2. What is the address of element at index 3?**

A) 512
B) 520
C) 508
D) 516

Q29. [Easy] **What is a Sparse Matrix?**

A) A matrix with mostly non-zero elements
B) A matrix in which the majority of elements are zero
C) A matrix with only 1 row
D) A square matrix with determinant 0

Q30. [Easy] **In 3-tuple representation of a sparse matrix, what do the three columns represent?**

A) Row index, Column index, Value of non-zero element
B) Base address, Offset, Length
C) Key, Left child, Right child
D) Row count, Column count, Total non-zero count

Q31. [Difficult] **An array A[0..5, 0..5] stores a tridiagonal matrix. How many non-zero diagonals does it contain, and what is the total number of non-zero elements?**

A) 3 diagonals, 16 elements
B) 3 diagonals, 18 elements
C) 2 diagonals, 12 elements
D) 1 diagonal, 6 elements

Q32. [Difficult] **What is the minimum number of comparisons needed to find BOTH the maximum and minimum elements in an array of size n?**

A) 2n - 2
B) ceil(3n / 2) - 2
C) n log n
D) n - 1

Q33. [Difficult] **In a 3D array A[1..2, 1..3, 1..4] stored in Row-Major order with B = 100, W = 1 byte, what is the address of A[2][2][3]?**

A) 118
B) 119
C) 120
D) 117

Q34. [Easy] **Which array operation takes O(1) time regardless of array size?**

A) Deleting by value
B) Accessing element by known index
C) Searching for an element in unsorted array
D) Inserting in sorted array

Q35. [Easy] **What is the major disadvantage of using an array to implement dynamic data structures?**

A) Slow access time
B) Fixed memory size requiring expensive array resizing and memory copy
C) More pointer overhead
D) Lack of indexing

Q36. [Medium] **An array of pointers in C is useful because:**

A) It allows efficient handling of variable-length strings without wasting memory
B) It makes CPU faster
C) It converts arrays to trees

D) It eliminates base address

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 2)

QID	Correct Option	Difficulty	Concept Tested
Q19	Option B	Medium	An array A[1..10, 1..15] is stored in Row-...
Q20	Option A	Medium	Given a 2D array A[1..10, 1..15] stored in...
Q21	Option B	Easy	What is the number of elements in a 2D arr...
Q22	Option C	Easy	Which programming language stores 2D array...
Q23	Option C	Easy	What is the time complexity to insert an e...
Q24	Option B	Easy	Why does an array provide O(1) random acce...
Q25	Option C	Medium	A triangular matrix of size n x n has non-...
Q26	Option B	Easy	If an array is declared as `int arr[5] = {...
Q27	Option C	Difficult	What is the time complexity of deleting an...
Q28	Option B	Easy	Given base address B = 500, element size W...
Q29	Option B	Easy	What is a Sparse Matrix?...
Q30	Option A	Easy	In 3-tuple representation of a sparse matr...
Q31	Option A	Difficult	An array A[0..5, 0..5] stores a tridiagona...
Q32	Option B	Difficult	What is the minimum number of comparisons ...
Q33	Option A	Difficult	In a 3D array A[1..2, 1..3, 1..4] stored i...
Q34	Option B	Easy	Which array operation takes O(1) time rega...
Q35	Option B	Easy	What is the major disadvantage of using an...
Q36	Option A	Medium	An array of pointers in C is useful becaus...

[INFO] Solution for Q19: Correct Option [B]

- Explanation: Formula: Address = B + W * [(i - LB1) * C + (j - LB2)]. Yahan B = 1000, W = 2, LB1 = 1, LB2 = 1, Columns C = 15 - 1 + 1 = 15. For A[4][6]: i = 4, j = 6. Address = 1000 + 2 * [(4 - 1) * 15 + (6 - 1)] = 1000 + 2 * [3 * 15 + 5] = 1000 + 2 * [45 + 5] = 1000 + 2 * 50 = 1000 + 100 = 1100.
- Exam Trap & Shortcut: Columns C = 15 use hoga, Rows R = 10 use nahi hoga!

[INFO] Solution for Q20: Correct Option [A]

- Explanation: Formula: Address = B + W * [(j - LB2) * R + (i - LB1)]. Yahan Rows R = 10 - 1 + 1 = 10. For A[4][6]: Address = 1000 + 2 * [(6 - 1) * 10 + (4 - 1)] = 1000 + 2 * [5 * 10 + 3] = 1000 + 2 * [53] = 1000 + 106 = 1106.
- Exam Trap & Shortcut: In Column-Major order, multiply column offset by number of ROWS (R = 10).

[INFO] Solution for Q21: Correct Option [B]

- Explanation: Rows = UB1 - LB1 + 1 = 5 - (-3) + 1 = 9. Columns = UB2 - LB2 + 1 = 8 - 2 + 1 = 7. Total elements = Rows * Columns = 9 * 7 = 63.
- Exam Trap & Shortcut: Subtracting without adding 1 gives (8 * 6 = 48) which is incorrect!

[INFO] Solution for Q22: Correct Option [C]

- Explanation: FORTRAN aur MATLAB column-major order use karte hain. C, C++, Python, aur Java row-major order use karte hain.
- Exam Trap & Shortcut: Standard C family languages are strictly Row-Major.

[INFO] Solution for Q23: Correct Option [C]

- Explanation: Beginning par insert karne ke liye saare existing n elements ko ek-ek position right shift karna padta hai, isliye time complexity O(n) hoti hai.
- Exam Trap & Shortcut: End par insert karna O(1) hota hai agar space available ho, lekin beginning par O(n) lagta hai.

[INFO] Solution for Q24: Correct Option [B]

- Explanation: Contiguous memory aur uniform data type hone ki wajah se index se direct memory address calculate ho jata hai: Base + i * W.
- Exam Trap & Shortcut: Sorting se random access ka koi lena dena nahi hai.

[INFO] Solution for Q25: Correct Option [C]

- Explanation: Row 1 has 1 element, Row 2 has 2 elements, ..., Row n has n elements. Total elements = 1 + 2 + ... + n = n(n + 1) / 2.
- Exam Trap & Shortcut: Strictly lower triangular (excluding diagonal) has n(n-1)/2, but including diagonal has n(n+1)/2.

[INFO] Solution for Q26: Correct Option [B]

- Explanation: C standard ke according, agar array partially initialize kiya jaye, toh remaining sabhi elements automatically 0 se initialize ho jaate hain.
- Exam Trap & Shortcut: Agar bilkul initialize na karein tab garbage value hoti hai, partial initialization par 0 hota hai.

[INFO] Solution for Q27: Correct Option [C]

- Explanation: Agar order preserve karna zaroori nahi hai, toh hum aakhiri element ko index i par copy karke array size ko 1 decrease kar sakte hain. Yeh constant time O(1) mein ho jata hai!
- Exam Trap & Shortcut: Standard deletion with order preservation takes O(n) due to shifting, but without order preservation it is O(1).

[INFO] Solution for Q28: Correct Option [B]

- Explanation: Formula: Loc(A[i]) = B + W * (i - LB) = 500 + 4 * (3 - (-2)) = 500 + 4 * (5) = 500 + 20 = 520.
- Exam Trap & Shortcut: Be careful with negative lower bound: 3 - (-2) = 5, not 1.

[INFO] Solution for Q29: Correct Option [B]

- Explanation: Sparse Matrix ek aisi matrix hoti hai jisme zero elements ki sankhya non-zero elements se bohot zyada hoti hai. Memory bachane ke liye isse 3-tuple format (Row, Col, Value) ya Linked List se represent karte hain.
- Exam Trap & Shortcut: Dense matrix has mostly non-zero values; Sparse matrix has mostly zero values.

[INFO] Solution for Q30: Correct Option [A]

- Explanation: 3-tuple array representation mein har row ek non-zero element ko store karti hai: (Row Index, Column Index, Value). First row store karti hai (Total Rows, Total Cols, Total Non-zeros).
- Exam Trap & Shortcut: Header row stores dimensions, subsequent rows store individual non-zero entries.

[INFO] Solution for Q31: Correct Option [A]

- Explanation: Tridiagonal matrix mein main diagonal, superdiagonal (above main), aur subdiagonal (below main) non-zero hote hain (total 3 diagonals). For size n = 6: Elements = n (main) + (n-1) (super) + (n-1) (sub) = 3n - 2 = 3(6) - 2 = 16 elements.
- Exam Trap & Shortcut: Formula for tridiagonal non-zero elements is 3n - 2.

[INFO] Solution for Q32: Correct Option [B]

- Explanation: Elements ko pairs mein compare karke tournament method se: n is even -> 1.5n - 2 comparisons. Best algorithm takes ceil(3n/2) - 2 comparisons, which is strictly better than the naive 2n - 2.
- Exam Trap & Shortcut: Naive linear search takes 2n - 2 comparisons. Tournament pair comparison takes 3n/2 - 2.

[INFO] Solution for Q33: Correct Option [A]

- Explanation: Dimensions: D1 = 2, D2 = 3, D3 = 4. Index: i1=2, i2=2, i3=3. Offset = (i1-1)*(D2*D3) + (i2-1)*(D3) + (i3-1) = (2-1)*(3*4) + (2-1)*(4) + (3-1) = 1*12 + 1*4 + 2 = 18. Address = 100 + 1 * 18 = 118.
- Exam Trap & Shortcut: In 3D Row-Major: multiply first index offset by product of remaining dimensions (D2 * D3).

[INFO] Solution for Q34: Correct Option [B]

- Explanation: Address computation Base + i * W takes constant CPU operations regardless of whether n is 10 or 10,000,000.
- Exam Trap & Shortcut: Searching takes O(n), access by index takes O(1).

[INFO] Solution for Q35: Correct Option [B]

- Explanation: Array ka size fixed hota hai. Jab array full ho jaye, toh naya bada array create karke saare elements copy karne padte hain (O(n) reallocation cost).
- Exam Trap & Shortcut: Pointers are used in linked lists, not static arrays.

[INFO] Solution for Q36: Correct Option [A]

- Explanation: Ragged arrays (array of pointers to char arrays) har string ke exact length ke barabar memory allocate karte hain, unlike 2D fixed char arrays jo rectangular padding waste karte hain.
- Exam Trap & Shortcut: Array of pointers saves memory for jagged/ragged data structures.

Unit 3: Stacks & Polish Notation Expressions (Q37 - Q54)

SECTION A: PRACTICE QUESTIONS

Q37. [Easy] Which data structure is used by the compiler to manage function calls and local variables?

A) Queue
B) Stack
C) Tree
D) Hash Table

Q38. [Medium] **What is the postfix evaluation of the expression: `5 3 + 8 2 - *` ?**

A) 48
B) 16
C) 32
D) 24

Q39. [Easy] **Convert the infix expression `A + B * C` to postfix notation:**

A) A B C * +
B) A B + C *
C) + A * B C
D) A B C + *

Q40. [Easy] **Which operator has right-to-left associativity?**

A) Addition (+)
B) Multiplication (*)
C) Exponentiation (^)
D) Division (/)

Q41. [Easy] **How many minimum stacks are required to implement a FIFO Queue?**

A) 1
B) 2
C) 3
D) 4

Q42. [Easy] **In an array implementation of a Stack of size MAX, the overflow condition occurs when:**

A) TOP == 0
B) TOP == MAX - 1
C) TOP == MAX
D) TOP == -1

Q43. [Easy] **What is the postfix form of the infix expression: `(A + B) * (C - D)` ?**

A) A B + C D - *
B) A B C D + - *
C) * + A B - C D
D) A B + * C D -

Q44. [Easy] **If the sequence of operations PUSH(1), PUSH(2), POP, PUSH(3), PUSH(4), POP, POP is executed, what is the sequence of popped elements?**

A) 2, 4, 3
B) 1, 2, 3
C) 2, 3, 4
D) 4, 3, 2

Q45. [Easy] **Which of the following problems can be solved using a Stack?**

A) Balanced parentheses checking
B) Decimal to Binary number conversion
C) Tower of Hanoi recursion
D) All of the above

Q46. [Medium] **What is the prefix form of the expression: `A + B * C` ?**

A) + A * B C
B) * + A B C
C) + * A B C
D) A B C * +

Q47. [Medium] **What is the result of evaluating the prefix expression: `+ * 2 3 4` ?**

A) 10
B) 14
C) 24
D) 12

Q48. [Medium] **How can two stacks be implemented efficiently in a single 1D array of size N?**

A) Stack 1 starts at index 0 and grows right; Stack 2 starts at index N - 1 and grows left
B) Alternate slots (even indices for Stack 1, odd for Stack 2)
C) Divide array exactly in half at N / 2
D) Impossible in a single array

Q49. [Medium] In the Min-Stack problem, how can we retrieve the minimum element in O(1) time?

A) By scanning the stack in O(n)
B) By maintaining an auxiliary stack that stores the minimum element seen so far at each level
C) By keeping the main stack sorted
D) By using a circular queue

Q50. [Easy] Which data structure is required to reverse a string of characters?

A) Queue
B) Stack
C) Tree
D) Hash Table

Q51. [Easy] What is the time complexity of PUSH and POP operations when a Stack is implemented using a Singly Linked List with head pointer?

A) O(1) and O(1)
B) O(1) and O(n)
C) O(n) and O(1)
D) O(n) and O(n)

Q52. [Easy] In an infix expression, which of the following is true regarding parentheses?

A) They override normal operator precedence
B) They are preserved in postfix notation
C) They are only allowed with addition
D) They must always be square brackets

Q53. [Easy] What is the postfix equivalent of `A + B + C` assuming left-to-right associativity of `+`?

A) A B + C +
B) A B C + +
C) + + A B C
D) A B + + C

Q54. [Easy] What will happen if you POP from an empty stack?

A) Stack Overflow
B) Stack Underflow error
C) Segmentation fault always
D) Stack doubles in size

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 3)

QID	Correct Option	Difficulty	Concept Tested
Q37	Option B	Easy	Which data structure is used by the compil...
Q38	Option A	Medium	What is the postfix evaluation of the expr...
Q39	Option A	Easy	Convert the infix expression `A + B * C` t...
Q40	Option C	Easy	Which operator has right-to-left associati...
Q41	Option B	Easy	How many minimum stacks are required to im...
Q42	Option B	Easy	In an array implementation of a Stack of s...
Q43	Option A	Easy	What is the postfix form of the infix expr...
Q44	Option A	Easy	If the sequence of operations PUSH(1), PUS...
Q45	Option D	Easy	Which of the following problems can be sol...
Q46	Option A	Medium	What is the prefix form of the expression:...
Q47	Option A	Medium	What is the result of evaluating the prefi...
Q48	Option A	Medium	How can two stacks be implemented efficien...
Q49	Option B	Medium	In the Min-Stack problem, how can we retri...
Q50	Option B	Easy	Which data structure is required to revers...
Q51	Option A	Easy	What is the time complexity of PUSH and PO...
Q52	Option A	Easy	In an infix expression, which of the follo...
Q53	Option A	Easy	What is the postfix equivalent of `A + B +...
Q54	Option B	Easy	What will happen if you POP from an empty ...

[INFO] Solution for Q37: Correct Option [B]

- Explanation: Function call stack (Call Stack / Runtime Stack) har function call ke Activation Record ko store karta hai. Yeh LIFO order follow karta hai.
- Exam Trap & Shortcut: CPU task scheduling uses Queue, but function execution uses Stack.

[INFO] Solution for Q38: Correct Option [A]

- Explanation: Step 1: Push 5, Push 3.
  Step 2: '+' -> pop 3, 5 -> 5 + 3 = 8 -> Push 8.
  Step 3: Push 8, Push 2.
  Step 4: '-' -> pop 2, 8 -> 8 - 2 = 6 -> Push 6.
  Step 5: '*' -> pop 6, 8 -> 8 * 6 = 48 -> Push 48. Answer = 48.
- Exam Trap & Shortcut: Ensure correct order during subtraction: first popped is right operand, second popped is left operand.

[INFO] Solution for Q39: Correct Option [A]

- Explanation: '*' has higher precedence than '+'. So `B * C` converts first to `B C *`. Then `A + (B C *)` becomes `A B C * +`.
- Exam Trap & Shortcut: Do not evaluate left to right without respecting operator precedence.

[INFO] Solution for Q40: Correct Option [C]

- Explanation: Exponentiation (^) right-to-left evaluate hota hai. E.g., `2 ^ 3 ^ 2` evaluates as `2 ^ (3 ^ 2) = 2 ^ 9 = 512`.
- Exam Trap & Shortcut: All standard arithmetic operators (+, -, *, /) are left-to-right associative, EXCEPT exponentiation.

[INFO] Solution for Q41: Correct Option [B]

- Explanation: Queue ko implement karne ke liye exactly 2 stacks ki zarurat hoti hai (Stack 1 for enqueue, Stack 2 for dequeue to invert the LIFO order to FIFO).
- Exam Trap & Shortcut: A single stack cannot reverse the order twice.

[INFO] Solution for Q42: Correct Option [B]

- Explanation: 0-based indexing mein array index 0 se MAX - 1 tak hota hai. Jab TOP == MAX - 1 ho jaye, stack full ho jata hai aur overflow condition aati hai.
- Exam Trap & Shortcut: TOP == MAX is an out-of-bounds index.

[INFO] Solution for Q43: Correct Option [A]

- Explanation: Parentheses evaluate first: `(A + B)` becomes `A B +`, `(C - D)` becomes `C D -`. Multiplication joins them: `A B + C D - *`.
- Exam Trap & Shortcut: Parentheses never appear in the final postfix expression.

[INFO] Solution for Q44: Correct Option [A]

- Explanation: Stack state trace:
  • Push 1 -> [1]
  • Push 2 -> [1, 2]
  • POP -> returns 2. Stack: [1]
  • Push 3 -> [1, 3]
  • Push 4 -> [1, 3, 4]
  • POP -> returns 4. Stack: [1, 3]
  • POP -> returns 3. Stack: [1]. Sequence = 2, 4, 3.
- Exam Trap & Shortcut: Follow stack LIFO carefully at each step.

[INFO] Solution for Q45: Correct Option [D]

- Explanation: Stack parentheses matching, base conversion (remainders reverse order mein print karne ke liye), aur recursive Tower of Hanoi sabhi ke liye use hota hai.
- Exam Trap & Shortcut: All three are classic standard stack applications.

[INFO] Solution for Q46: Correct Option [A]

- Explanation: First evaluate `B * C` -> `* B C`. Then `A + (* B C)` -> `+ A * B C`.
- Exam Trap & Shortcut: In Prefix (Polish Notation), the operator comes BEFORE operands.

[INFO] Solution for Q47: Correct Option [A]

- Explanation: Prefix ko right-to-left scan karte hain:
  Operands: 4, 3, 2. Operator `*` applies to 2 and 3 -> 2 * 3 = 6. Then operator `+` applies to 6 and 4 -> 6 + 4 = 10.
- Exam Trap & Shortcut: Prefix expressions are evaluated by scanning from RIGHT to LEFT.

[INFO] Solution for Q48: Correct Option [A]

- Explanation: Stack 1 starts at 0 (top1 = -1) aur right move karta hai. Stack 2 starts at N-1 (top2 = N) aur left move karta hai. Overflow tabhi hoga jab dono tops meet karein (`top1 + 1 == top2`), maximizing memory utilization.
- Exam Trap & Shortcut: Dividing in half at N/2 causes false overflow when one stack is full and the other is empty.

[INFO] Solution for Q49: Correct Option [B]

- Explanation: Ek auxiliary stack maintain kiya jata hai. Har push par, naye element aur current min ka minimum auxiliary stack mein push hota hai. `getMin()` simply auxiliary stack ke top ko constant time O(1) mein return karta hai.
- Exam Trap & Shortcut: Sorting the stack would violate O(1) push and pop operations.

[INFO] Solution for Q50: Correct Option [B]

- Explanation: Characters ko sequence mein stack mein PUSH karo, fir POP karke nikaloge toh LIFO property ki wajah se exact reverse order milega.
- Exam Trap & Shortcut: Queue maintains original FIFO order, so it cannot reverse.

[INFO] Solution for Q51: Correct Option [A]

- Explanation: Linked list ke HEAD par insert (Push) aur HEAD se delete (Pop) dono O(1) constant time operations hote hain.
- Exam Trap & Shortcut: If push/pop were implemented at the tail of a singly linked list without tail pointer, it would be O(n).

[INFO] Solution for Q52: Correct Option [A]

- Explanation: Parentheses highest priority hold karte hain aur default operator precedence ko override karte hain.
- Exam Trap & Shortcut: Parentheses never appear in postfix or prefix expressions.

[INFO] Solution for Q53: Correct Option [A]

- Explanation: Due to left associativity: `(A + B) + C` -> `A B + + C` -> `A B + C +`.
- Exam Trap & Shortcut: If right-associative, it would be A B C + +.

[INFO] Solution for Q54: Correct Option [B]

- Explanation: Empty stack se element remove karne ki koshish ko Stack Underflow kehte hain.
- Exam Trap & Shortcut: Overflow happens on inserting into a full stack; Underflow on removing from an empty stack.

Unit 4: Queues & Circular Queues (Q55 - Q72)

SECTION A: PRACTICE QUESTIONS

Q55. [Medium] In a circular queue of capacity N implemented using an array, the condition for queue FULL is:

A) (REAR + 1) % N == FRONT
B) REAR == FRONT
C) REAR == N - 1
D) FRONT == (REAR + 1)

Q56. [Easy] What major drawback of a linear array-based queue is solved by a circular queue?

A) Slow search time
B) False overflow (unused free spaces at front)
C) Lack of dynamic memory
D) Inability to insert integers

Q57. [Difficult] In a circular queue with size N = 5, FRONT = 2 and REAR = 1. How many elements are currently in the queue?

A) 4
B) 1
C) 5
D) 3

Q58. [Medium] A double-ended queue (Deque) that allows insertion at both ends but deletion only at one end is called:

A) Input-restricted deque
B) Output-restricted deque
C) Priority queue
D) Circular queue

Q59. [Easy] Which data structure is typically used to implement Breadth-First Search (BFS) traversal of a graph?

A) Stack
B) Queue
C) Priority Queue
D) Binary Search Tree

Q60. [Easy] How many queues are required to implement a LIFO Stack?

A) 1
B) 2
C) 3
D) 4

Q61. [Easy] In a priority queue, how are elements dequeued?

A) Strictly in the order they arrived
B) According to their associated priority values
C) In random order
D) LIFO order

Q62. [Easy] What is the time complexity of ENQUEUE and DEQUEUE operations in an array-based circular queue?

A) O(1) and O(1)
B) O(1) and O(n)
C) O(n) and O(1)
D) O(n) and O(n)

Q63. [Easy] If FRONT = -1 and REAR = -1 in a circular queue, the queue is:

A) Full
B) Empty
C) Containing exactly 1 element
D) Overflowed

Q64. [Easy] Which scheduling algorithm in Operating Systems directly uses a FIFO Queue?

A) First-Come, First-Served (FCFS)
B) Shortest Job First (SJF)
C) Priority Scheduling
D) Multilevel Feedback Queue

Q65. [Easy] What is an input-restricted deque?

A) Insertion at one end, deletion at both ends
B) Deletion at one end, insertion at both ends
C) Insertion and deletion at both ends
D) Only allows positive inputs

Q66. [Medium] In a queue implemented using a Singly Linked List, where should insertion and deletion take place for O(1) time efficiency?

A) Insert at REAR (tail), Delete at FRONT (head)
B) Insert at FRONT, Delete at REAR
C) Both at FRONT
D) Both at REAR

Q67. [Difficult] What happens when a circular queue of size 6 with FRONT = 3 and REAR = 2 tries to enqueue an element?

A) Successful insertion at index 3
B) Queue Overflow error
C) FRONT becomes 4
D) REAR becomes 4

Q68. [Easy] **Which data structure is most suitable for Round-Robin CPU scheduling?**

A) Circular Queue
B) Priority Queue
C) Binary Search Tree
D) Stack

Q69. [Medium] **If FRONT = 0 and REAR = 0 in a circular queue, how many elements are in the queue?**

A) Exactly 0 elements (Empty)
B) Exactly 1 element
C) Full queue
D) Indeterminate without flags

Q70. [Easy] **What is the time complexity to find the minimum element in a standard FIFO Queue of n elements?**

A) O(1)
B) O(log n)
C) O(n)
D) O(n^2)

Q71. [Easy] **In a queue, deletion is also referred to as:**

A) PUSH
B) POP / DEQUEUE
C) PEEK
D) ENQUEUE

Q72. [Easy] **Which underlying structure gives O(1) insertion and deletion at both ends for a Deque?**

A) Doubly Linked List
B) Singly Linked List without tail
C) Linear Array with fixed front
D) Binary Tree

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 4)

QID	Correct Option	Difficulty	Concept Tested
Q55	Option A	Medium	In a circular queue of capacity N implemen...
Q56	Option B	Easy	What major drawback of a linear array-base...
Q57	Option A	Difficult	In a circular queue with size N = 5, FRONT...
Q58	Option B	Medium	A double-ended queue (Deque) that allows i...
Q59	Option B	Easy	Which data structure is typically used to ...
Q60	Option B	Easy	How many queues are required to implement ...
Q61	Option B	Easy	In a priority queue, how are elements dequ...
Q62	Option A	Easy	What is the time complexity of ENQUEUE and...
Q63	Option B	Easy	If FRONT = -1 and REAR = -1 in a circular ...
Q64	Option A	Easy	Which scheduling algorithm in Operating Sy...
Q65	Option A	Easy	What is an input-restricted deque?...
Q66	Option A	Medium	In a queue implemented using a Singly Link...
Q67	Option B	Difficult	What happens when a circular queue of size...
Q68	Option A	Easy	Which data structure is most suitable for ...
Q69	Option B	Medium	If FRONT = 0 and REAR = 0 in a circular qu...
Q70	Option C	Easy	What is the time complexity to find the mi...
Q71	Option B	Easy	In a queue, deletion is also referred to a...
Q72	Option A	Easy	Which underlying structure gives O(1) inse...

[INFO] Solution for Q55: Correct Option [A]

- Explanation: Circular queue mein jab agla rear index front ke barabar ho jaye, toh queue full hoti hai: `(REAR + 1) % N == FRONT`.
- Exam Trap & Shortcut: REAR == FRONT empty condition ke liye use hota hai ya 1-element condition ke liye, full ke liye nahi.

[INFO] Solution for Q56: Correct Option [B]

- Explanation: Linear queue mein jab elements delete hote hain, front aage badh jata hai. Rear end par pahunchne ke baad naye elements insert nahi ho pate bhale hi aage slots khali hon (False Overflow). Circular queue modulo wrap se isko solve karta hai.
- Exam Trap & Shortcut: Circular queue does not change O(1) time complexity; it saves memory space.

[INFO] Solution for Q57: Correct Option [A]

- Explanation: Formula: Count = (REAR - FRONT + N) % N. If count == 0 and queue is full, count = N. Here: (1 - 2 + 5) % 5 = 4 % 5 = 4 elements. Elements are located at indices 2, 3, 4, 0.
- Exam Trap & Shortcut: REAR < FRONT means the queue has wrapped around the circular boundary!

[INFO] Solution for Q58: Correct Option [B]

- Explanation: Output-restricted deque mein deletion sirf ek end se hota hai, jabki insertion dono ends se allow hota hai.
- Exam Trap & Shortcut: Input-restricted deque mein insertion ek end se hota hai aur deletion dono ends se hota hai.

[INFO] Solution for Q59: Correct Option [B]

- Explanation: BFS level-by-level traversal karta hai, jiske liye FIFO order zaroori hai. Isliye standard Queue use hoti hai.
- Exam Trap & Shortcut: DFS uses Stack; BFS uses Queue.

[INFO] Solution for Q60: Correct Option [B]

- Explanation: Stack ko simulate karne ke liye 2 queues ki zarurat hoti hai.
- Exam Trap & Shortcut: Just like 2 stacks make a queue, 2 queues make a stack.

[INFO] Solution for Q61: Correct Option [B]

- Explanation: Priority queue mein highest (ya lowest, depending on min/max heap) priority wala element sabse pehle dequeue hota hai.
- Exam Trap & Shortcut: It does not strictly follow FIFO.

[INFO] Solution for Q62: Correct Option [A]

- Explanation: Dono operations pointer arithmetic (modulo N) se instant hote hain, isliye dono O(1) constant time lete hain.
- Exam Trap & Shortcut: No element shifting is performed in a circular queue.

[INFO] Solution for Q63: Correct Option [B]

- Explanation: FRONT == -1 aur REAR == -1 standard empty (underflow) condition hoti hai.
- Exam Trap & Shortcut: When 1 element is added, FRONT and REAR both become 0.

[INFO] Solution for Q64: Correct Option [A]

- Explanation: FCFS CPU scheduling strictly FIFO queue principles par chalta hai.
- Exam Trap & Shortcut: SJF and Priority scheduling use Priority Queues (Min-Heaps).

[INFO] Solution for Q65: Correct Option [A]

- Explanation: Input-restricted deque mein insertion sirf ek end se allow hota hai, lekin deletion dono ends (front aur rear) se ho sakta hai.
- Exam Trap & Shortcut: Output-restricted has deletion at one end and insertion at both.

[INFO] Solution for Q66: Correct Option [A]

- Explanation: Head se delete karna O(1) hota hai. Tail pointer ke saath end par insert karna O(1) hota hai. Singly linked list mein tail se delete karna O(n) hota hai (prev node dhoondna padta hai). Isliye Insert at Tail and Delete at Head is optimal.
- Exam Trap & Shortcut: Deleting at the tail of a singly linked list is O(n), so deletion MUST be at the head.

[INFO] Solution for Q67: Correct Option [B]

- Explanation: Check full condition: `(REAR + 1) % N == (2 + 1) % 6 = 3 % 6 = 3`. Yahan FRONT == 3 hai, so `(REAR + 1) % N == FRONT` is TRUE. Queue already full hai, isliye Queue Overflow hoga!
- Exam Trap & Shortcut: REAR = FRONT - 1 in modulo N means full!

[INFO] Solution for Q68: Correct Option [A]

- Explanation: Round Robin mein processes time quantum ke basis par cyclic order mein execute hoti hain. Circular Queue expired process ko pichhe bhej kar agle process ko instant run karne ke liye best hai.
- Exam Trap & Shortcut: Priority queue is for Priority Scheduling, Circular queue is for Round Robin.

[INFO] Solution for Q69: Correct Option [B]

- Explanation: Standard convention mein jab pehla element insert hota hai, FRONT aur REAR dono 0 par point karte hain. So there is exactly 1 element in the queue.
- Exam Trap & Shortcut: Empty state is usually represented by FRONT = -1.

[INFO] Solution for Q70: Correct Option [C]

- Explanation: Queue unsorted hoti hai aur sirf FRONT se access deti hai. Minimum find karne ke liye saare n elements ko dequeue karke check karna padega (O(n) time).
- Exam Trap & Shortcut: Queue does not keep elements sorted like a Min-Heap.

[INFO] Solution for Q71: Correct Option [B]

- Explanation: Queue mein insert ko ENQUEUE aur delete ko DEQUEUE kehte hain.
- Exam Trap & Shortcut: PUSH and POP are stack terminology.

[INFO] Solution for Q72: Correct Option [A]

- Explanation: Doubly Linked List mein HEAD aur TAIL dono ends par insert aur delete constant time O(1) mein ho jata hai.
- Exam Trap & Shortcut: Singly linked list cannot delete from the tail in O(1).

Unit 5: Linked Lists (Q73 - Q90)

SECTION A: PRACTICE QUESTIONS

Q73. [Easy] What is the time complexity to insert a new node at the beginning of a Singly Linked List of length n?

A) O(1)
B) O(n)
C) O(log n)
D) O(n^2)

Q74. [Medium] In a Singly Linked List with only a HEAD pointer, what is the time complexity to delete the LAST node?

A) O(1)
B) O(n)
C) O(log n)
D) O(n^2)

Q75. [Easy] How does a Circular Linked List differ from a standard Singly Linked List?

A) It has no data fields
B) The next pointer of the last node points back to the first node instead of NULL
C) It can only store numbers
D) It uses two pointers per node

Q76. [Medium] How many pointers are changed when inserting a node between two existing nodes in a Doubly Linked List?

A) 2
B) 3
C) 4
D) 1

Q77. [Medium] How many minimum pointers are required to reverse a Singly Linked List iteratively in a single pass?

A) 1
B) 2
C) 3
D) 4

Q78. [Easy] Floyd's Cycle-Finding Algorithm (Hare and Tortoise) to detect a loop in a linked list uses two pointers moving at speeds:

A) Both move 1 step
B) Slow moves 1 step, Fast moves 2 steps
C) Slow moves 1 step, Fast moves 3 steps
D) Both move 2 steps

Q79. [Easy] Which of the following is an advantage of a Linked List over an Array?

A) Random access to elements
B) Dynamic size and efficient insertions/deletions without shifting
C) Lower memory usage per element
D) Better cache locality

Q80. [Easy] A linked list where each node points to both its successor and its predecessor, and the last node connects to the first, is called:

A) Circular Doubly Linked List
B) Singly Linked List
C) Header Linked List
D) Self-referential array

Q81. [Easy] What does the term "self-referential structure" mean in the context of linked list nodes in C?

A) A structure that refers to standard library functions
B) A structure containing a pointer member that points to the same structure type
C) A structure that cannot be modified
D) A structure with only one variable

Q82. [Medium] What happens if you execute `free(head)` before `head = head->next` during node deletion in C?

A) Normal deletion
B) Memory leak and dangling pointer bug (loss of rest of the list)
C) Automatic traversal
D) Compile time error

Q83. [Medium] **How can you find the middle element of a Singly Linked List in a single pass?**

A) Traverse twice to count total nodes
B) Use two pointers: slow moves 1 step, fast moves 2 steps. When fast reaches end, slow is at the middle
C) Middle cannot be found in a single pass
D) Use binary search

Q84. [Easy] **In a Circular Linked List with pointer `last` pointing to the last node, what is the address of the first node?**

A) last
B) last->next
C) NULL
D) last->prev

Q85. [Easy] **What is a Header Linked List?**

A) A list that has a special header node at the very beginning storing metadata like list length
B) A list with no pointers
C) A list stored in ROM
D) An array of headers

Q86. [Easy] **What is the time complexity to merge two sorted Singly Linked Lists of lengths m and n into a single sorted list?**

A) O(m * n)
B) O(m + n)
C) O(log(m + n))
D) O(1)

Q87. [Medium] **What is the space overhead of a Doubly Linked List with n nodes compared to a Singly Linked List on a 64-bit system?**

A) 4n bytes
B) 8n bytes (one extra 8-byte pointer per node)
C) No extra overhead
D) 16n bytes

Q88. [Easy] **Which linked list allows traversal in both directions without any NULL pointer?**

A) Singly Linked List
B) Circular Doubly Linked List
C) Standard Doubly Linked List
D) Header Linked List

Q89. [Medium] **If a node `p` is in a doubly linked list, how do you remove `p` assuming it is not head or tail?**

A) p->prev->next = p->next; p->next->prev = p->prev; free(p);
B) p->next = p->prev; free(p);
C) free(p); p->prev->next = p->next;
D) p = p->next;

Q90. [Easy] **Why can Binary Search NOT be efficiently executed on a Singly Linked List?**

A) Linked list cannot be sorted
B) Calculating and accessing the middle element in a linked list requires O(n) sequential traversal
C) Pointers cannot be compared
D) Memory is continuous

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 5)

QID	Correct Option	Difficulty	Concept Tested
Q73	Option A	Easy	What is the time complexity to insert a ne...
Q74	Option B	Medium	In a Singly Linked List with only a HEAD p...
Q75	Option B	Easy	How does a Circular Linked List differ fro...
Q76	Option C	Medium	How many pointers are changed when inserti...
Q77	Option C	Medium	How many minimum pointers are required to ...
Q78	Option B	Easy	Floyd's Cycle-Finding Algorithm (Hare and ...
Q79	Option B	Easy	Which of the following is an advantage of ...
Q80	Option A	Easy	A linked list where each node points to bo...
Q81	Option B	Easy	What does the term "self-referential struc...
Q82	Option B	Medium	What happens if you execute `free(head)` b...
Q83	Option B	Medium	How can you find the middle element of a S...
Q84	Option B	Easy	In a Circular Linked List with pointer `la...
Q85	Option A	Easy	What is a Header Linked List?...
Q86	Option B	Easy	What is the time complexity to merge two s...
Q87	Option B	Medium	What is the space overhead of a Doubly Lin...
Q88	Option B	Easy	Which linked list allows traversal in both...
Q89	Option A	Medium	If a node `p` is in a doubly linked list, ...
Q90	Option B	Easy	Why can Binary Search NOT be efficiently e...

[INFO] Solution for Q73: Correct Option [A]

- Explanation: Head par insert karne ke liye sirf naye node ka next pointer HEAD par set karna hota hai aur HEAD ko update karna hota hai. Yeh O(1) constant time operation hai.
- Exam Trap & Shortcut: Array mein beginning insert O(n) hota hai shifting ki wajah se, lekin linked list mein O(1) hota hai.

[INFO] Solution for Q74: Correct Option [B]

- Explanation: Last node ko delete karne ke liye hume second-to-last node tak traverse karna padta hai taaki uska next pointer NULL kiya ja sake. Is traversal mein O(n) time lagta hai.
- Exam Trap & Shortcut: Even if a TAIL pointer is given in Singly Linked List, deleting the last node still takes O(n) because we cannot move backward to find the second-to-last node!

[INFO] Solution for Q75: Correct Option [B]

- Explanation: Circular linked list mein koi NULL pointer nahi hota. Last node ka next pointer first (HEAD) node ko point karta hai, forming a circle.
- Exam Trap & Shortcut: Two pointers per node is the characteristic of a Doubly Linked List, not a Circular Singly Linked List.

[INFO] Solution for Q76: Correct Option [C]

- Explanation: Total 4 pointers change hote hain:
  1. newNode->next = nextNode
  2. newNode->prev = prevNode
  3. prevNode->next = newNode
  4. nextNode->prev = newNode.
- Exam Trap & Shortcut: Students often forget the backward pointers and answer 2.

[INFO] Solution for Q77: Correct Option [C]

- Explanation: Iterative reversal ke liye 3 pointers chahiye hote hain: `prev` (initially NULL), `curr` (initially HEAD), aur `next` (to store the rest of the list before breaking the link).
- Exam Trap & Shortcut: Attempting with 2 pointers causes loss of reference to the remaining chain.

[INFO] Solution for Q78: Correct Option [B]

- Explanation: Slow pointer 1 step aage badhta hai aur Fast pointer 2 steps. Agar cycle hoti hai, toh dono pointers loop ke andar kisi na kisi node par meet karte hain. Time: O(n), Space: O(1).
- Exam Trap & Shortcut: If they move at the same speed, fast will never catch slow.

[INFO] Solution for Q79: Correct Option [B]

- Explanation: Linked list runtime par dynamically grow/shrink kar sakti hai aur bina elements shift kiye O(1) time mein insert/delete kar sakti hai.
- Exam Trap & Shortcut: Arrays have better cache locality and zero pointer memory overhead.

[INFO] Solution for Q80: Correct Option [A]

- Explanation: Dono directions mein movement + circular wrap-around is called Circular Doubly Linked List.
- Exam Trap & Shortcut: Standard DLL is not circular.

[INFO] Solution for Q81: Correct Option [B]

- Explanation: `struct Node { int data; struct Node *next; };` - yahan `next` pointer usi structure type ko point karta hai, isliye isse self-referential structure kehte hain.
- Exam Trap & Shortcut: It contains a pointer to itself, not an actual embedded copy of itself.

[INFO] Solution for Q82: Correct Option [B]

- Explanation: Agar `head` ko pehle free kar doge, toh `head->next` access karna undefined behavior ho jayega aur bachi hui list ka address kho jayega (Memory Leak).
- Exam Trap & Shortcut: Always save `head->next` in a temporary pointer before calling free().

[INFO] Solution for Q83: Correct Option [B]

- Explanation: Two-pointer (Runner technique): Fast pointer slow pointer se double speed par chalta hai. Jab fast end of list par pahunchta hai, slow pointer theek middle node par hota hai.
- Exam Trap & Shortcut: Binary search is impossible on linked lists due to lack of random access.

[INFO] Solution for Q84: Correct Option [B]

- Explanation: Circular list mein last node ka agla pointer pehle node ko point karta hai: `last->next` is the first node.
- Exam Trap & Shortcut: Maintaining a `last` pointer gives O(1) access to both first (`last->next`) and last node!

[INFO] Solution for Q85: Correct Option [A]

- Explanation: Header linked list mein first node ek dummy header hota hai jo actual data list ka part nahi hota balki list length, max value jaisi metadata store karta hai.
- Exam Trap & Shortcut: Header node simplifies insertion/deletion edge cases at the beginning.

[INFO] Solution for Q86: Correct Option [B]

- Explanation: Dono lists ke head elements ko compare karke pointers rewire karte hue O(m + n) time mein merge kiya ja sakta hai with O(1) extra space.
- Exam Trap & Shortcut: Merge sort merge step on linked list takes linear O(m+n) time.

[INFO] Solution for Q87: Correct Option [B]

- Explanation: 64-bit systems par har pointer 8 bytes leta hai. Doubly linked list har node mein ek extra `prev` pointer rakhti hai, so extra memory = 8 * n bytes.
- Exam Trap & Shortcut: Pointer size on 64-bit systems is 8 bytes, on 32-bit it is 4 bytes.

[INFO] Solution for Q88: Correct Option [B]

- Explanation: Circular Doubly Linked List has prev and next pointers in all nodes, and wraps around circularly at both ends with no NULL.
- Exam Trap & Shortcut: Standard DLL has NULL at both ends.

[INFO] Solution for Q89: Correct Option [A]

- Explanation: Node p ko bypass karne ke liye uske predecessor ke next ko p->next aur successor ke prev ko p->prev set karte hain, fir p ko free karte hain.
- Exam Trap & Shortcut: Freeing p before updating pointers causes dangling pointers and crashes.

[INFO] Solution for Q90: Correct Option [B]

- Explanation: Binary Search requires O(1) random access to find `mid`. Linked list mein middle tak pahunchne ke liye hi O(n) lag jata hai, ruining the O(log n) efficiency.
- Exam Trap & Shortcut: Binary Search requires random access (arrays), not sequential access.

Unit 6: Recursion & Recurrences (Q91 - Q105)

SECTION A: PRACTICE QUESTIONS

Q91. [Easy] What is the base case in a recursive algorithm?

A) The case that calls the function again
B) The condition under which recursion terminates without making further recursive calls
C) The largest input value
D) The main function

Q92. [Medium] **What is the solution to the recurrence relation: T(n) = 2T(n/2) + O(n) ?**

A) O(n)
B) O(n log n)
C) O(n^2)
D) O(log n)

Q93. [Easy] **What is the solution to the recurrence relation: T(n) = T(n - 1) + O(1) with T(1) = 1?**

A) O(1)
B) O(log n)
C) O(n)
D) O(n^2)

Q94. [Easy] **How many moves are required to solve the Tower of Hanoi problem with n disks?**

A) 2n - 1
B) 2^n - 1
C) n^2
D) 2^n + 1

Q95. [Medium] **What is tail recursion?**

A) Recursion where the recursive call is the very last statement executed in the function
B) Recursion that has two base cases
C) Recursion that occurs at the beginning of the function
D) Recursion that takes O(n^2) space

Q96. [Easy] **What is the auxiliary stack space complexity of recursive Binary Search on an array of size n?**

A) O(1)
B) O(log n)
C) O(n)
D) O(n log n)

Q97. [Easy] **What does the following recursive function return for n = 4?**

```
int fun(int n) {
   if (n <= 1) return 1;
   return n + fun(n - 1);
}
```

A) 10
B) 24
C) 7
D) 15

Q98. [Easy] **Which data structure can be used to convert any recursive algorithm into an iterative one?**

A) Queue
B) Stack
C) Array
D) Hash Table

Q99. [Medium] **What is the time complexity of the naive recursive Fibonacci algorithm: `fib(n) = fib(n-1) + fib(n-2)`?**

A) O(n)
B) O(2^n)
C) O(n^2)
D) O(log n)

Q100. [Easy] **What is indirect recursion?**

A) A function calling itself directly
B) Function A calling Function B, and Function B calling Function A
C) A loop inside a function
D) A function with no return value

Q101. [Medium] **What is the solution of the recurrence: T(n) = 2T(n/2) + O(1)?**

A) O(n)
B) O(log n)
C) O(n log n)
D) O(1)

Q102. [Easy] **What is the solution of the recurrence: T(n) = T(n/2) + O(1) with T(1) = 1?**

A) O(1)
B) O(log n)
C) O(n)

D) O(n log n)

Q103. [Medium] What is the maximum depth of the recursion tree for Quick Sort in the worst case?

A) O(log n)
B) O(n)
C) O(1)
D) O(n^2)

Q104. [Easy] What error occurs if a recursive function never reaches its base case?

A) CPU Overheating
B) Stack Overflow runtime error
C) Compile time syntax error
D) Array bounds error

Q105. [Easy] What does the following recursive function return for fun(5)?

```
int fun(int n) {
   if (n <= 1) return 1;
   return n * fun(n - 1);
}
```

A) 15
B) 120
C) 24
D) 60

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 6)

QID	Correct Option	Difficulty	Concept Tested
Q91	Option B	Easy	What is the base case in a recursive algor...
Q92	Option B	Medium	What is the solution to the recurrence rel...
Q93	Option C	Easy	What is the solution to the recurrence rel...
Q94	Option B	Easy	How many moves are required to solve the T...
Q95	Option A	Medium	What is tail recursion?...
Q96	Option B	Easy	What is the auxiliary stack space complexi...
Q97	Option A	Easy	What does the following recursive function...
Q98	Option B	Easy	Which data structure can be used to conver...
Q99	Option B	Medium	What is the time complexity of the naive r...
Q100	Option B	Easy	What is indirect recursion?...
Q101	Option A	Medium	What is the solution of the recurrence: T(...
Q102	Option B	Easy	What is the solution of the recurrence: T(...
Q103	Option B	Medium	What is the maximum depth of the recursion...
Q104	Option B	Easy	What error occurs if a recursive function ...
Q105	Option B	Easy	What does the following recursive function...

[INFO] Solution for Q91: Correct Option [B]

- Explanation: Base case terminating condition hoti hai jo infinite recursion aur Stack Overflow se bachati hai.
- Exam Trap & Shortcut: Without a base case, recursion causes stack overflow.

[INFO] Solution for Q92: Correct Option [B]

- Explanation: By Master Theorem: a = 2, b = 2, f(n) = n. Since n^(log_b a) = n^(log2 2) = n^1 = f(n), this falls under Case 2 of Master Theorem. Hence T(n) = O(n log n). This is the Merge Sort recurrence.
- Exam Trap & Shortcut: Recognize this immediately as standard Merge Sort recurrence.

[INFO] Solution for Q93: Correct Option [C]

- Explanation: T(n) = T(n-1) + 1 = T(n-2) + 1 + 1 = ... = T(1) + (n - 1) = 1 + n - 1 = n. So T(n) = O(n).
- Exam Trap & Shortcut: Decreasing recurrence by 1 with constant work solves to O(n).

[INFO] Solution for Q94: Correct Option [B]

- Explanation: Recurrence: T(n) = 2T(n-1) + 1. Solving gives T(n) = 2^n - 1 moves. For n = 3, moves = 2^3 - 1 = 7.
- Exam Trap & Shortcut: 2n - 1 is linear, 2^n - 1 is exponential.

[INFO] Solution for Q95: Correct Option [A]

- Explanation: Tail recursion mein recursive call ke baad koi work perform nahi hota. Modern compilers tail call optimization (TCO) karke isse constant stack space O(1) mein run kar sakte hain.
- Exam Trap & Shortcut: If any calculation happens after recursive call (like n * fact(n-1)), it is NOT tail recursive.

[INFO] Solution for Q96: Correct Option [B]

- Explanation: Har recursive call array ko half karti hai, isliye maximum call stack depth log2(n) hoti hai. Space = O(log n). (Iterative binary search takes O(1) space).
- Exam Trap & Shortcut: Iterative binary search is O(1) space, but recursive binary search is O(log n) space due to call stack.

[INFO] Solution for Q97: Correct Option [A]

- Explanation: fun(4) = 4 + fun(3) = 4 + 3 + fun(2) = 4 + 3 + 2 + fun(1) = 4 + 3 + 2 + 1 = 10. (Sum of first n natural numbers).
- Exam Trap & Shortcut: It is addition (+), not multiplication (*) like factorial.

[INFO] Solution for Q98: Correct Option [B]

- Explanation: Recursion internally system call stack use karta hai. Isliye explicit Stack data structure use karke kisi bhi recursive function ko iterative banaya ja sakta hai.
- Exam Trap & Shortcut: Queue gives BFS traversal, Stack simulates recursion.

[INFO] Solution for Q99: Correct Option [B]

- Explanation: Har call do nayi recursive calls branch karti hai without memoization, creating a binary recursion tree of depth n. Time complexity is exponential O(2^n) (more precisely O(1.618^n)).
- Exam Trap & Shortcut: Dynamic programming reduces this to O(n), but naive recursion is exponential O(2^n).

[INFO] Solution for Q100: Correct Option [B]

- Explanation: Direct recursion mein function khud ko directly call karta hai. Indirect recursion mein ek cycle hoti hai: A calls B, B calls A.
- Exam Trap & Shortcut: Mutual recursion is also known as indirect recursion.

[INFO] Solution for Q101: Correct Option [A]

- Explanation: By Master Theorem: a = 2, b = 2, f(n) = O(1) = n^0. Here n^(log_b a) = n^(log2 2) = n^1. Since n^1 > n^0, Case 1 applies: T(n) = Theta(n^(log_b a)) = Theta(n).
- Exam Trap & Shortcut: Do not confuse with Merge sort where f(n) is n. Here f(n) is 1, so answer is O(n).

[INFO] Solution for Q102: Correct Option [B]

- Explanation: This is the classic Binary Search recurrence relation. Halving the problem size with constant comparison work solves to O(log n).
- Exam Trap & Shortcut: Standard Binary Search recurrence.

[INFO] Solution for Q103: Correct Option [B]

- Explanation: Worst case mein pivot har baar 1 aur n-1 elements ka partition banata hai. Isse recursion depth n tak chali jaati hai, taking O(n) stack memory.
- Exam Trap & Shortcut: Average depth is O(log n), but worst-case depth is O(n).

[INFO] Solution for Q104: Correct Option [B]

- Explanation: Infinite recursive calls continuously system call stack par activation records push karti rehti hain jab tak allocated stack memory exhaust nahi ho jaati (Stack Overflow).
- Exam Trap & Shortcut: It is a runtime crash, not a compilation error.

[INFO] Solution for Q105: Correct Option [B]

- Explanation: This is the standard factorial function: 5! = 5 * 4 * 3 * 2 * 1 = 120.
- Exam Trap & Shortcut: 5! = 120.

Unit 7: Trees & Binary Tree Properties (Q106 - Q125)

SECTION A: PRACTICE QUESTIONS

Q106. [Easy] In any non-empty binary tree, if n0 is the number of leaf nodes and n2 is the number of nodes with degree 2, then:

A) n0 = n2
B) n0 = n2 + 1
C) n0 = n2 - 1
D) n0 = 2 * n2

Q107. [Easy] What is the maximum number of nodes in a binary tree of height h (assuming root is at height 0)?

A) 2^h
B) 2^(h+1) - 1
C) 2^(h-1)
D) 2^h - 1

Q108. [Easy] A Full Binary Tree has 15 leaf nodes. How many internal nodes with degree 2 does it have?

A) 14
B) 15
C) 16
D) 7

Q109. [Medium] What is the minimum height of a binary tree containing 100 nodes (root at height 0)?

A) 6
B) 7
C) 5
D) 8

Q110. [Medium] **How many structurally distinct binary trees can be formed with n = 3 unlabelled nodes?**

A) 3
B) 4
C) 5
D) 6

Q111. [Easy] **Which two traversals are sufficient to uniquely construct a binary tree?**

A) Preorder and Postorder
B) Inorder and Preorder
C) Level order and Postorder
D) Preorder and Level order

Q112. [Medium] **Given Inorder traversal: `D B E A F C` and Preorder traversal: `A B D E C F`. What is the Postorder traversal?**

A) D E B F C A
B) D E B C F A
C) E D B F C A
D) D B E F C A

Q113. [Easy] **In a complete binary tree stored in an array with 1-based indexing, where is the parent of node at index i located?**

A) 2 * i
B) 2 * i + 1
C) floor(i / 2)
D) i - 1

Q114. [Easy] **A binary tree in which every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible, is called:**

A) Full Binary Tree
B) Complete Binary Tree
C) Perfect Binary Tree
D) Balanced Tree

Q115. [Easy] **What is the maximum number of nodes at level k of a binary tree (root at level 0)?**

A) 2^k
B) 2^(k-1)
C) 2^(k+1)
D) 2k

Q116. [Easy] **What is the degree of a leaf node in any tree?**

A) 1
B) 0
C) 2
D) -1

Q117. [Easy] **A tree with 25 vertices has how many edges?**

A) 25
B) 24
C) 50
D) 12

Q118. [Medium] **How many distinct binary search trees (BSTs) can be created with 4 distinct keys?**

A) 14
B) 24
C) 5
D) 42

Q119. [Easy] **In which tree traversal is the root node visited FIRST?**

A) Inorder
B) Preorder
C) Postorder
D) Euler tour

Q120. [Easy] **In which tree traversal is the root node visited LAST?**

A) Inorder
B) Preorder
C) Postorder
D) Level order

Q121. [Medium] **A full binary tree has 20 internal nodes. What is the total number of nodes in the tree?**

A) 40
B) 41
C) 39
D) 21

Q122. [Easy] **What is the height of a degenerate (completely skewed) binary tree with n nodes?**

A) log2(n)
B) n - 1
C) n / 2
D) 2^n

Q123. [Easy] **In a binary tree, an ancestor of node X is defined as:**

A) Any node on the unique path from root to X (excluding X)
B) Only the immediate parent of X
C) Any leaf node
D) Siblings of X

Q124. [Easy] **If every internal node of a tree has at most m children, the tree is called an:**

A) Binary tree
B) m-ary (or multiway) tree
C) AVL tree
D) Bipartite tree

Q125. [Difficult] **Can a binary tree be uniquely reconstructed if ONLY Preorder and Postorder traversals are given?**

A) Yes, always
B) No, unless the tree is known to be a Full Binary Tree
C) Never possible
D) Only if all keys are negative

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 7)

QID	Correct Option	Difficulty	Concept Tested
Q106	Option B	Easy	In any non-empty binary tree, if n0 is the...
Q107	Option B	Easy	What is the maximum number of nodes in a b...
Q108	Option A	Easy	A Full Binary Tree has 15 leaf nodes. How ...
Q109	Option A	Medium	What is the minimum height of a binary tre...
Q110	Option C	Medium	How many structurally distinct binary tree...
Q111	Option B	Easy	Which two traversals are sufficient to uni...
Q112	Option A	Medium	Given Inorder traversal: `D B E A F C` and...
Q113	Option C	Easy	In a complete binary tree stored in an arr...
Q114	Option B	Easy	A binary tree in which every level, except...
Q115	Option A	Easy	What is the maximum number of nodes at lev...
Q116	Option B	Easy	What is the degree of a leaf node in any t...
Q117	Option B	Easy	A tree with 25 vertices has how many edges...
Q118	Option A	Medium	How many distinct binary search trees (BST...
Q119	Option B	Easy	In which tree traversal is the root node v...
Q120	Option C	Easy	In which tree traversal is the root node v...
Q121	Option B	Medium	A full binary tree has 20 internal nodes. ...
Q122	Option B	Easy	What is the height of a degenerate (comple...
Q123	Option A	Easy	In a binary tree, an ancestor of node X is...
Q124	Option B	Easy	If every internal node of a tree has at mo...
Q125	Option B	Difficult	Can a binary tree be uniquely reconstructe...

[INFO] Solution for Q106: Correct Option [B]

- Explanation: Standard Binary Tree Theorem: n0 = n2 + 1. Leaf nodes ki sankhya hamesha degree-2 nodes se exactly 1 zyada hoti hai.
- Exam Trap & Shortcut: This holds for ANY binary tree, not just full or complete binary trees.

[INFO] Solution for Q107: Correct Option [B]

- Explanation: Height 0 par 1 node (2^1 - 1 = 1). Height h par total nodes = 2^0 + 2^1 + ... + 2^h = 2^(h+1) - 1.
- Exam Trap & Shortcut: If root is defined at height 1, formula is 2^h - 1. CUET PG standard assumes root at height 0, giving 2^(h+1) - 1.

[INFO] Solution for Q108: Correct Option [A]

- Explanation: Formula: n0 = n2 + 1. Given n0 = 15 => 15 = n2 + 1 => n2 = 14.
- Exam Trap & Shortcut: Simple direct formula application: n2 = n0 - 1.

[INFO] Solution for Q109: Correct Option [A]

- Explanation: Min height formula: h_min = ceil(log2(n + 1)) - 1. For n = 100: n + 1 = 101. log2(101) ~= 6.658 -> ceil is 7. Height = 7 - 1 = 6. (Check: height 6 can hold up to 2^7 - 1 = 127 nodes; height 5 can only hold 63 nodes).
- Exam Trap & Shortcut: Height 5 can only hold 63 nodes, so 100 nodes require height 6.

[INFO] Solution for Q110: Correct Option [C]

- Explanation: Catalan Number C3 = (1 / (3 + 1)) * (6 choose 3) = (1 / 4) * (20) = 5 distinct binary trees.
- Exam Trap & Shortcut: For labelled nodes it would be C_n * n! = 5 * 6 = 30, but for unlabelled nodes it is strictly 5.

[INFO] Solution for Q111: Correct Option [B]

- Explanation: Unique binary tree construct karne ke liye INORDER traversal ke saath PREORDER ya POSTORDER ka hona mandatory hai. Preorder + Postorder cannot guarantee a unique tree.
- Exam Trap & Shortcut: Inorder is compulsory because it tells what is on the left and right of the root.

[INFO] Solution for Q112: Correct Option [A]

- Explanation: From Preorder, root is A.
  In Inorder, left subtree is `D B E`, right subtree is `F C`.
  In left subtree `D B E`, Preorder says B is root, D is left, E is right.
  In right subtree `F C`, Preorder says C is root, F is left.
  Postorder (Left, Right, Root): Left of A gives `D E B`, Right of A gives `F C`, Root is A.
  Combined Postorder: `D E B F C A`.
- Exam Trap & Shortcut: Root is always at the end in Postorder traversal.

[INFO] Solution for Q113: Correct Option [C]

- Explanation: For 1-based indexing:
  Parent(i) = floor(i / 2)
  Left Child(i) = 2 * i
  Right Child(i) = 2 * i + 1.
- Exam Trap & Shortcut: In 0-based indexing, Parent is floor((i - 1) / 2).

[INFO] Solution for Q114: Correct Option [B]

- Explanation: This is the exact formal definition of a Complete Binary Tree, which is used to implement Binary Heaps.
- Exam Trap & Shortcut: Perfect binary tree requires even the last level to be completely full.

[INFO] Solution for Q115: Correct Option [A]

- Explanation: Level 0 has 2^0 = 1 node. Level 1 has 2^1 = 2 nodes. Level k has 2^k nodes.
- Exam Trap & Shortcut: At level k, max nodes is 2^k.

[INFO] Solution for Q116: Correct Option [B]

- Explanation: Degree of a node is the number of children it has. Leaf nodes have 0 children, so their degree is 0.
- Exam Trap & Shortcut: Do not confuse tree node degree (number of children) with graph vertex degree (number of incident edges).

[INFO] Solution for Q117: Correct Option [B]

- Explanation: Any tree with n vertices has exactly n - 1 edges: 25 - 1 = 24.
- Exam Trap & Shortcut: Direct property: |E| = |V| - 1 for any tree.

[INFO] Solution for Q118: Correct Option [A]

- Explanation: Catalan number C4 = (1 / 5) * (8 choose 4) = (1 / 5) * (70) = 14 distinct BSTs.
- Exam Trap & Shortcut: C1 = 1, C2 = 2, C3 = 5, C4 = 14, C5 = 42.

[INFO] Solution for Q119: Correct Option [B]

- Explanation: Preorder traversal visits Node (Root) first, then Left Subtree, then Right Subtree (NLR).
- Exam Trap & Shortcut: Postorder visits root last; Inorder visits root in the middle.

[INFO] Solution for Q120: Correct Option [C]

- Explanation: Postorder visits Left Subtree, Right Subtree, and finally the Node (Root) (LRN).
- Exam Trap & Shortcut: Post means after/last.

[INFO] Solution for Q121: Correct Option [B]

- Explanation: In a full binary tree with I internal nodes: Leaf nodes L = I + 1 = 20 + 1 = 21. Total nodes N = I + L = 20 + 21 = 41. (Or direct formula: N = 2I + 1 = 2(20) + 1 = 41).
- Exam Trap & Shortcut: Total nodes in a full binary tree with I internal nodes is always 2I + 1.

[INFO] Solution for Q122: Correct Option [B]

- Explanation: Degenerate tree basically ek linear linked list ban jata hai, jahan har node ka sirf 1 child hota hai. Height = n - 1.
- Exam Trap & Shortcut: Degenerate tree has worst-case height and O(n) search time.

[INFO] Solution for Q123: Correct Option [A]

- Explanation: Root se node X tak ke path par jitne bhi nodes aate hain (excluding X), woh sabhi node X ke ancestors hote hain.
- Exam Trap & Shortcut: Immediate parent is just one ancestor; all nodes above X on the path to root are ancestors.

[INFO] Solution for Q124: Correct Option [B]

- Explanation: Generalization of binary trees (where m = 2) to m children is called an m-ary tree.
- Exam Trap & Shortcut: When m = 3, it is ternary; general m is m-ary.

[INFO] Solution for Q125: Correct Option [B]

- Explanation: General binary tree cannot be reconstructed uniquely without Inorder. BUT if the tree is strictly Full (every node has 0 or 2 children), then Preorder + Postorder can uniquely construct it!
- Exam Trap & Shortcut: Standard answer is NO for general binary trees, but for FULL binary trees it is YES.

Unit 8: Binary Search Trees & AVL Trees (Q126 - Q145)

SECTION A: PRACTICE QUESTIONS

Q126. [Easy] The Inorder traversal of a Binary Search Tree (BST) always produces keys in:

A) Random order
B) Strictly ascending (sorted) order
C) Strictly descending order
D) Level-wise order

Q127. [Easy] **What is the worst-case time complexity of searching a key in a Binary Search Tree of n nodes?**

A) O(log n)
B) O(n)
C) O(n log n)
D) O(1)

Q128. [Easy] **What is the allowed range of Balance Factor for any node in an AVL Tree?**

A) {0, 1}
B) {-1, 0, 1}
C) {-2, 0, 2}
D) Any integer

Q129. [Medium] **When a new node is inserted into the right subtree of the left child, causing an imbalance, which rotation is required?**

A) LL Rotation
B) RR Rotation
C) LR Rotation
D) RL Rotation

Q130. [Difficult] **What is the minimum number of nodes in an AVL Tree of height h = 3 (where height of single node is 0)?**

A) 4
B) 7
C) 8
D) 10

Q131. [Easy] **When deleting a node with two children in a BST, it is replaced by its:**

A) Root node
B) Inorder Successor or Inorder Predecessor
C) Leftmost child of tree
D) Any leaf node

Q132. [Easy] **What is the worst-case time complexity of search, insertion, and deletion in an AVL Tree with n nodes?**

A) O(n)
B) O(log n)
C) O(n^2)
D) O(1)

Q133. [Easy] **If the Preorder traversal of a BST is: `30, 20, 10, 25, 40, 35, 50`, what is its Inorder traversal?**

A) 10, 20, 25, 30, 35, 40, 50
B) 50, 40, 35, 30, 25, 20, 10
C) 30, 20, 40, 10, 25, 35, 50
D) Cannot be determined without Postorder

Q134. [Easy] **An AVL tree has how many single rotations and double rotations respectively?**

A) 2 single (LL, RR) and 2 double (LR, RL)
B) 4 single and 0 double
C) 1 single and 3 double
D) 0 single and 4 double

Q135. [Easy] **In a BST, the maximum key is always located at:**

A) The root node
B) The rightmost node
C) The leftmost node
D) Any leaf node

Q136. [Medium] **What is the maximum height of an AVL tree with n nodes?**

A) log2(n)
B) 1.44 log2(n)
C) n - 1
D) 2 * log2(n)

Q137. [Difficult] **Which of the following is true regarding AVL Trees compared to Red-Black Trees?**

A) AVL trees are more rigidly balanced, making search faster but insertion/deletion slower due to more rotations
B) Red-Black trees are strictly more balanced than AVL trees
C) AVL trees require more storage space for colors
D) AVL trees have O(n) search time

Q138. [Easy] **What is the Balance Factor of a leaf node in any AVL tree?**

A) 0
B) 1
C) -1
D) 2

Q139. [Medium] **In a BST, what is the Inorder Successor of a node with a non-empty right subtree?**

A) The minimum node in its right subtree (leftmost child of right subtree)
B) Its parent node
C) The maximum node in its left subtree
D) The root node

Q140. [Easy] **Inserting keys into a BST in strictly descending order results in what shape of tree?**

A) Perfect binary tree
B) Left-skewed binary tree
C) Right-skewed binary tree
D) Full binary tree

Q141. [Medium] **What is the minimum number of nodes in an AVL tree of height h = 4?**

A) 12
B) 16
C) 20
D) 15

Q142. [Easy] **Which rotation is performed when a node is inserted into the left subtree of the left child?**

A) Single Right Rotation (LL Rotation)
B) Single Left Rotation (RR Rotation)
C) Double LR Rotation
D) Double RL Rotation

Q143. [Easy] **Can a BST contain duplicate keys in its standard textbook implementation?**

A) No, all keys must be distinct
B) Yes, duplicates are allowed in both subtrees simultaneously
C) Only at the root
D) Yes, but only in red-black trees

Q144. [Difficult] **How many rotations are required in the WORST case after an insertion in an AVL tree?**

A) At most 1 single or double rotation
B) O(log n) rotations
C) O(n) rotations
D) Exactly 4 rotations

Q145. [Easy] **What is the time complexity to find the minimum element in an AVL tree with n nodes?**

A) O(1)
B) O(log n)
C) O(n)
D) O(n log n)

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 8)

QID	Correct Option	Difficulty	Concept Tested
Q126	Option B	Easy	The Inorder traversal of a Binary Search T...
Q127	Option B	Easy	What is the worst-case time complexity of ...
Q128	Option B	Easy	What is the allowed range of Balance Facto...
Q129	Option C	Medium	When a new node is inserted into the right...
Q130	Option B	Difficult	What is the minimum number of nodes in an ...
Q131	Option B	Easy	When deleting a node with two children in ...
Q132	Option B	Easy	What is the worst-case time complexity of ...
Q133	Option A	Easy	If the Preorder traversal of a BST is: `30...
Q134	Option A	Easy	An AVL tree has how many single rotations ...
Q135	Option B	Easy	In a BST, the maximum key is always locate...
Q136	Option B	Medium	What is the maximum height of an AVL tree ...
Q137	Option A	Difficult	Which of the following is true regarding A...
Q138	Option A	Easy	What is the Balance Factor of a leaf node ...
Q139	Option A	Medium	In a BST, what is the Inorder Successor of...
Q140	Option B	Easy	Inserting keys into a BST in strictly desc...
Q141	Option A	Medium	What is the minimum number of nodes in an ...
Q142	Option A	Easy	Which rotation is performed when a node is...
Q143	Option A	Easy	Can a BST contain duplicate keys in its st...
Q144	Option A	Difficult	How many rotations are required in the WOR...
Q145	Option B	Easy	What is the time complexity to find the mi...

[INFO] Solution for Q126: Correct Option [B]

- Explanation: BST property: Left < Root < Right. Inorder visits Left, Root, Right, which guarantees keys in strictly ascending sorted order.
- Exam Trap & Shortcut: This is the most celebrated property of BSTs; use it to identify BSTs in MCQ options!

[INFO] Solution for Q127: Correct Option [B]

- Explanation: Worst case tab hota hai jab tree completely skewed ban jaye (e.g., elements inserted in already sorted order). Skewed BST linked list ki tarah behave karta hai, taking O(n) time.
- Exam Trap & Shortcut: Average case is O(log n), but worst case is O(n).

[INFO] Solution for Q128: Correct Option [B]

- Explanation: AVL tree self-balancing BST hota hai jisme har node ka balance factor BF = Height(Left) - Height(Right) strictly {-1, 0, +1} hona chahiye.
- Exam Trap & Shortcut: If BF becomes +2 or -2, rotations are immediately triggered.

[INFO] Solution for Q129: Correct Option [C]

- Explanation: Left child (L) ke Right subtree (R) mein insert hone se LR imbalance banta hai. Iske liye LR double rotation (first left rotate on child, then right rotate on parent) ki zarurat hoti hai.
- Exam Trap & Shortcut: LR imbalance requires Left then Right rotation.

[INFO] Solution for Q130: Correct Option [B]

- Explanation: Recurrence: N(h) = N(h-1) + N(h-2) + 1. Base cases: N(0) = 1, N(1) = 2.
  N(2) = N(1) + N(0) + 1 = 2 + 1 + 1 = 4.
  N(3) = N(2) + N(1) + 1 = 4 + 2 + 1 = 7 nodes.
- Exam Trap & Shortcut: For height 3, minimum nodes is 7, maximum is 2^(3+1) - 1 = 15.

[INFO] Solution for Q131: Correct Option [B]

- Explanation: Two children waale node ko delete karte time BST property maintain rakhne ke liye use ya toh Inorder Predecessor (maximum of left subtree) ya Inorder Successor (minimum of right subtree) se replace karte hain.
- Exam Trap & Shortcut: Replacing with arbitrary nodes violates the BST property.

[INFO] Solution for Q132: Correct Option [B]

- Explanation: Kyunki AVL tree hamesha balanced rehta hai (height bounded by 1.44 log2 n), iske search, insert, aur delete operations sabhi worst case mein bhi strictly O(log n) hote hain.
- Exam Trap & Shortcut: Unlike regular BST which degrades to O(n), AVL guarantees O(log n).

[INFO] Solution for Q133: Correct Option [A]

- Explanation: Trick question! Kisi bhi BST ka Inorder traversal hamesha elements ka sorted order hota hai. Bas diye gaye elements ko sort kar do: `10, 20, 25, 30, 35, 40, 50`!
- Exam Trap & Shortcut: Do not waste time drawing the tree; just sort the elements in ascending order!

[INFO] Solution for Q134: Correct Option [A]

- Explanation: LL aur RR single rotations hain (1 rotation around root). LR aur RL double rotations hain (2 rotations: child then root).
- Exam Trap & Shortcut: LR and RL are composite double rotations.

[INFO] Solution for Q135: Correct Option [B]

- Explanation: BST property ke mutabiq bade elements hamesha right mein hote hain, isliye root se lagaatar right pointers follow karne par rightmost node maximum element hota hai.
- Exam Trap & Shortcut: Minimum element leftmost node par hota hai.

[INFO] Solution for Q136: Correct Option [B]

- Explanation: AVL tree height formula: h <= 1.4404 * log2(n + 2) - 0.328 ~= 1.44 log2 n.
- Exam Trap & Shortcut: Red-Black tree has maximum height 2 log2(n + 1); AVL is tighter at 1.44 log2 n.

[INFO] Solution for Q137: Correct Option [A]

- Explanation: AVL trees strictly height balanced hoti hain (BF <= 1), isliye AVL mein search faster hota hai. Red-Black trees thodi loose hoti hain (height <= 2 log n), jisse insertion/deletion mein kam rotations lagte hain.
- Exam Trap & Shortcut: AVL is preferred for lookup-intensive systems; Red-Black for insert/delete-intensive systems.

[INFO] Solution for Q138: Correct Option [A]

- Explanation: Leaf node ka left child bhi NULL hota hai (height -1) aur right child bhi NULL (height -1). BF = (-1) - (-1) = 0.
- Exam Trap & Shortcut: Leaf node has balance factor exactly 0.

[INFO] Solution for Q139: Correct Option [A]

- Explanation: Inorder successor means the immediately next larger element. Agar right subtree exist karta hai, toh right subtree ka sabse chhota element (leftmost node) hi Inorder Successor hota hai.
- Exam Trap & Shortcut: If right subtree is empty, Inorder successor is the deepest ancestor whose left child is also an ancestor.

[INFO] Solution for Q140: Correct Option [B]

- Explanation: Har naya key pichhle key se chhota hoga, isliye har naya element left child banta chala jayega (Left-Skewed Tree).
- Exam Trap & Shortcut: Ascending order creates Right-Skewed; Descending creates Left-Skewed.

[INFO] Solution for Q141: Correct Option [A]

- Explanation: N(0)=1, N(1)=2, N(2)=4, N(3)=7, N(4) = N(3) + N(2) + 1 = 7 + 4 + 1 = 12 nodes.
- Exam Trap & Shortcut: Follow recurrence: N(h) = N(h-1) + N(h-2) + 1.

[INFO] Solution for Q142: Correct Option [A]

- Explanation: LL case: Left child ke Left subtree mein insertion causes right-leaning push, which is balanced by a Single Right Rotation around the unbalanced node.
- Exam Trap & Shortcut: LL imbalance is resolved by RIGHT rotation; RR imbalance by LEFT rotation.

[INFO] Solution for Q143: Correct Option [A]

- Explanation: Standard BST formal definition specifies strict inequalities: Left < Root < Right. Duplicates are either disallowed or handled with a counter in the node.
- Exam Trap & Shortcut: If duplicates are allowed, a convention (e.g. <= left) must be consistently maintained.

[INFO] Solution for Q144: Correct Option [A]

- Explanation: Beautiful property of AVL trees: Single insertion requires AT MOST ONE rotation (either single or double) to restore balance to the entire tree! Deletion, however, may require up to O(log n) rotations along the path to root.
- Exam Trap & Shortcut: Insertion needs at most 1 rotation; Deletion may propagate O(log n) rotations.

[INFO] Solution for Q145: Correct Option [B]

- Explanation: Leftmost leaf tak traverse karna hota hai. Since AVL tree height is strictly bounded by O(log n), finding minimum takes O(log n) time.
- Exam Trap & Shortcut: In regular skewed BST it can take O(n), but in AVL it is strictly O(log n).

Unit 9: B-Trees & B+ Trees (Q146 - Q163)

SECTION A: PRACTICE QUESTIONS

Q146. [Easy] In a B-Tree of order m, what is the maximum number of keys an internal node can hold?

A) m
B) m - 1
C) ceil(m / 2)
D) 2m

Q147. [Medium] In a B-Tree of order 5, what is the minimum number of keys that a non-root internal node must contain?

A) 1
B) 2
C) 3
D) 4

Q148. [Easy] Where are data records and pointers stored in a B+ Tree?

A) In internal nodes only
B) In all nodes equally
C) ONLY in the leaf nodes
D) In the root node only

Q149. [Easy] Why are leaf nodes linked together as a linked list in a B+ Tree?

A) To reduce height
B) To enable extremely efficient range queries and sequential scanning
C) To prevent deletion
D) To store negative numbers

Q150. [Medium] In a B-Tree of order m, what is the minimum number of children the ROOT node can have (if not a leaf)?

A) 1
B) 2
C) ceil(m / 2)
D) m - 1

Q151. [Easy] All leaf nodes in a B-Tree are located at:

A) Different levels depending on balance
B) The EXACT same level (depth)
C) Level 1
D) Alternating levels

Q152. [Medium] Why are B-Trees preferred over Binary Search Trees for disk-based databases?

A) B-Trees have shorter height and larger node capacity, minimizing expensive disk I/O operations
B) BST uses too much RAM
C) B-Trees don't need sorting
D) B-Trees are faster in CPU cache

Q153. [Easy] When a node overflows in a B-Tree during insertion, what operation is performed?

A) Rotation
B) Node Splitting and pushing the median key up to the parent
C) Deletion of root
D) Rehashing

Q154. [Easy] In a B-Tree of order 4 (2-3-4 Tree), what is the maximum number of children an internal node can have?

A) 2
B) 3
C) 4
D) 5

Q155. [Medium] In a B-Tree of order 4, what are the possible numbers of keys an internal non-root node can contain?

A) 1, 2, or 3 keys
B) Exactly 4 keys
C) 2 or 3 keys
D) 0 keys

Q156. [Medium] What happens when a key is deleted from a leaf in a B-Tree and causes underflow?

A) Tree crashes
B) The node borrows a key from an immediate sibling or merges with a sibling
C) The root is automatically deleted

D) Tree converts to BST

Q157. [Easy] In a B+ Tree, search keys present in internal nodes:

A) Never appear in leaf nodes
B) MUST also appear in leaf nodes along with their associated data records
C) Are encrypted
D) Store actual file contents

Q158. [Easy] What is the order of growth for search time in a B-Tree with n keys and order m?

A) O(log_m n)
B) O(n)
C) O(m * n)
D) O(n^2)

Q159. [Easy] Which popular database storage engine uses B+ Trees for primary clustered indexing?

A) MySQL InnoDB
B) Redis
C) Memcached
D) Apache Kafka

Q160. [Difficult] If a B-Tree of order m = 3 has 3 levels (root at level 1), what is the maximum number of keys it can hold?

A) 8
B) 26
C) 27
D) 13

Q161. [Difficult] What distinguishes a B* Tree from a standard B-Tree?

A) B* Tree internal nodes must be at least 2/3 full instead of 1/2 full
B) B* Tree has no leaves
C) B* Tree stores characters only
D) B* Tree is unbalanced

Q162. [Medium] Can a B-Tree grow taller from the leaves?

A) Yes, by adding leaves at the bottom
B) No, B-Trees grow upwards towards the root when the root splits
C) B-Trees never change height
D) Only when elements are deleted

Q163. [Medium] In a B-Tree of order 7, what is the minimum number of children for a non-root internal node?

A) 3
B) 4
C) 5
D) 6

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 9)

QID	Correct Option	Difficulty	Concept Tested
Q146	Option B	Easy	In a B-Tree of order m, what is the maximu...
Q147	Option B	Medium	In a B-Tree of order 5, what is the minimu...
Q148	Option C	Easy	Where are data records and pointers stored...
Q149	Option B	Easy	Why are leaf nodes linked together as a li...
Q150	Option B	Medium	In a B-Tree of order m, what is the minimu...
Q151	Option B	Easy	All leaf nodes in a B-Tree are located at:...
Q152	Option A	Medium	Why are B-Trees preferred over Binary Sear...
Q153	Option B	Easy	When a node overflows in a B-Tree during i...
Q154	Option C	Easy	In a B-Tree of order 4 (2-3-4 Tree), what ...
Q155	Option A	Medium	In a B-Tree of order 4, what are the possi...
Q156	Option B	Medium	What happens when a key is deleted from a ...
Q157	Option B	Easy	In a B+ Tree, search keys present in inter...
Q158	Option A	Easy	What is the order of growth for search tim...
Q159	Option A	Easy	Which popular database storage engine uses...
Q160	Option B	Difficult	If a B-Tree of order m = 3 has 3 levels (r...
Q161	Option A	Difficult	What distinguishes a B* Tree from a standa...
Q162	Option B	Medium	Can a B-Tree grow taller from the leaves?...
Q163	Option B	Medium	In a B-Tree of order 7, what is the minimu...

[INFO] Solution for Q146: Correct Option [B]

- Explanation: Order m ka matlab max children = m. Ek node mein keys ki sankhya hamesha children se 1 kam hoti hai: Max Keys = m - 1.
- Exam Trap & Shortcut: Max children is m, but Max keys is m - 1.

[INFO] Solution for Q147: Correct Option [B]

- Explanation: Min keys formula = ceil(m / 2) - 1. For m = 5: ceil(5 / 2) - 1 = ceil(2.5) - 1 = 3 - 1 = 2 keys.
- Exam Trap & Shortcut: ceil(5/2) = 3 gives the minimum CHILDREN; subtract 1 to get minimum KEYS (3 - 1 = 2).

[INFO] Solution for Q148: Correct Option [C]

- Explanation: B+ Tree mein actual data pointers sirf aur sirf LEAF NODES mein store hote hain. Internal nodes sirf router/index keys store karte hain.
- Exam Trap & Shortcut: In standard B-Trees, data is stored in both internal and leaf nodes.

[INFO] Solution for Q149: Correct Option [B]

- Explanation: Database queries jaise `WHERE age BETWEEN 20 AND 30` ke liye B+ tree direct leaf linked list ko linearly traverse kar leta hai bina root par wapas jaye.
- Exam Trap & Shortcut: Range search efficiency is the primary reason databases use B+ Trees over B-Trees.

[INFO] Solution for Q150: Correct Option [B]

- Explanation: Root node ke liye special exception hota hai: Root must have AT LEAST 2 children (unless the tree has only 1 node). It does NOT need to satisfy the ceil(m/2) rule.
- Exam Trap & Shortcut: Internal nodes require ceil(m/2) children, but the ROOT only requires at least 2 children.

[INFO] Solution for Q151: Correct Option [B]

- Explanation: B-Tree hamesha perfectly height balanced hoti hai: Saare leaf nodes exact same depth par hote hain.
- Exam Trap & Shortcut: Leaves can never be at different levels in a B-Tree.

[INFO] Solution for Q152: Correct Option [A]

- Explanation: Disk block reading bohot slow hoti hai. B-Tree ek hi disk block mein multiple keys fit kar deta hai, jisse tree ki height bohot kam hoti hai (2-3 levels for millions of records), minimizing disk reads.
- Exam Trap & Shortcut: Disk access time is the bottleneck, not CPU cycles.

[INFO] Solution for Q153: Correct Option [B]

- Explanation: Jab node mein m keys ho jaati hain, node split hota hai do halves mein aur median key parent node mein push ho jaati hai.
- Exam Trap & Shortcut: B-Trees grow from the leaves towards the root via node splitting.

[INFO] Solution for Q154: Correct Option [C]

- Explanation: Order m = 4 means maximum number of children is m = 4.
- Exam Trap & Shortcut: Order defines maximum children.

[INFO] Solution for Q155: Correct Option [A]

- Explanation: Min keys = ceil(4 / 2) - 1 = 2 - 1 = 1 key. Max keys = 4 - 1 = 3 keys. So node can hold 1, 2, or 3 keys.
- Exam Trap & Shortcut: A 2-3-4 tree node can have 1, 2, or 3 keys (having 2, 3, or 4 children).

[INFO] Solution for Q156: Correct Option [B]

- Explanation: Underflow handle karne ke liye pehle sibling se borrow (key redistribution) ki koshish karte hain. Agar sibling ke paas bhi minimum keys hon, toh dono siblings parent key ke saath merge ho jaate hain.
- Exam Trap & Shortcut: Borrow first, merge if borrowing is impossible.

[INFO] Solution for Q157: Correct Option [B]

- Explanation: B+ Tree internal nodes serve only as routers/indices. Every search key must ultimately reside in a leaf node where the actual record pointer is stored.
- Exam Trap & Shortcut: Key duplication between internal and leaf nodes is an inherent feature of B+ Trees.

[INFO] Solution for Q158: Correct Option [A]

- Explanation: Tree height is proportional to log base m of n. Search takes O(log_m n) disk operations.
- Exam Trap & Shortcut: Since m is large (often 100 to 1000 in DBs), log_m n is extremely small (2 to 4).

[INFO] Solution for Q159: Correct Option [A]

- Explanation: MySQL InnoDB, PostgreSQL, aur Oracle tables primary index ke liye B+ Trees use karte hain.
- Exam Trap & Shortcut: Redis is an in-memory hash/skiplist store; InnoDB is standard B+ Tree.

[INFO] Solution for Q160: Correct Option [B]

- Explanation: Max keys formula: m^h - 1. For m = 3, h = 3: 3^3 - 1 = 27 - 1 = 26 keys. (Level 1: 2 keys, Level 2: 3 nodes * 2 = 6 keys, Level 3: 9 nodes * 2 = 18 keys. Total = 2 + 6 + 18 = 26).
- Exam Trap & Shortcut: Formula is m^h - 1, not m^h.

[INFO] Solution for Q161: Correct Option [A]

- Explanation: B* Tree node fill factor ko increase karta hai: Non-root nodes must be at least 2/3 full (floor((2m-1)/3) keys), reducing node splits and increasing disk space utilization.
- Exam Trap & Shortcut: Standard B-Tree is 1/2 full; B* Tree is 2/3 full.

[INFO] Solution for Q162: Correct Option [B]

- Explanation: B-Tree hamesha leaves par insert karta hai. Jab root full ho kar split hota hai, tab ek naya root banta hai aur tree ki height 1 level badhti hai (Grows from root upwards).
- Exam Trap & Shortcut: Binary trees grow downwards; B-Trees grow upwards!

[INFO] Solution for Q163: Correct Option [B]

- Explanation: Min children = ceil(m / 2) = ceil(7 / 2) = ceil(3.5) = 4 children.
- Exam Trap & Shortcut: ceil(7/2) is 4.

Unit 10: Hashing & Collision Resolution (Q164 - Q182)

SECTION A: PRACTICE QUESTIONS

Q164. [Easy] What is the Load Factor (alpha) of a Hash Table with n keys and table size m?

A) m / n
B) n / m
C) n * m
D) n + m

Q165. [Easy] Which collision resolution technique suffers from Primary Clustering?

A) Separate Chaining
B) Linear Probing
C) Quadratic Probing
D) Double Hashing

Q166. [Medium] What is the probe sequence formula in Double Hashing for probe attempt i?

A) (h1(k) + i * h2(k)) % m
B) (h1(k) * h2(k) + i) % m
C) (h1(k) + i^2) % m
D) (h1(k) + h2(k)) % m

Q167. [Medium] In Open Addressing hashing, what is the maximum possible value of the Load Factor alpha?

A) alpha can be any value > 1
B) alpha <= 1
C) alpha must be 0
D) alpha = 2

Q168. [Medium] Given hash function h(k) = k mod 7. Keys 15, 22, 8 are inserted using Linear Probing in an array of size 7 (indices 0..6). What is the index of key 8?

A) 1
B) 2
C) 3
D) 4

Q169. [Easy] What is the worst-case search time complexity in a Hash Table of size m containing n keys implemented using Separate Chaining?

A) O(1)
B) O(log n)
C) O(n)
D) O(m)

Q170. [Easy] What is the desirable property of a good hash function?

A) Minimizes collisions by distributing keys uniformly across the table
B) Maps all keys to index 0
C) Is as complex and slow as possible
D) Produces negative indices

Q171. [Difficult] Secondary clustering in quadratic probing occurs because:

A) Keys that hash to different positions follow the same probe sequence
B) Keys that hash to the same initial position follow the exact same probe sequence
C) The table size is prime
D) Elements are deleted

Q172. [Medium] Why is the table size m usually chosen as a prime number in the division hash method h(k) = k mod m?

A) To make modulo calculation faster
B) To reduce patterns and common factors between keys and table size, minimizing collisions
C) To allow negative numbers
D) Because computers prefer prime numbers

Q173. [Easy] In separate chaining, what data structure is maintained at each slot of the hash table?

A) Stack
B) Linked List
C) Binary Tree
D) Circular Array

Q174. [Easy] In Quadratic Probing, the probe sequence is defined as:

A) (h(k) + i) % m
B) (h(k) + c1*i + c2*i^2) % m
C) (h(k) * i^2) % m
D) (h(k) + 2^i) % m

Q175. [Medium] What is Rehashing?

A) Re-running the hash function twice
B) Dynamically allocating a larger hash table (typically double the size) and re-inserting all existing keys when load factor exceeds a threshold
C) Deleting the hash table
D) Using two hash functions

Q176. [Difficult] How is deletion typically handled in an Open Addressing hash table using Linear Probing?

A) Directly setting the slot to NULL/empty
B) Marking the slot with a special "DELETED" (tombstone) flag so subsequent searches do not terminate prematurely
C) Shifting all subsequent elements left
D) Deletion is impossible

Q177. [Easy] In the Mid-Square hash method, if key k = 45, what is the hash code using the middle two digits of k^2?

A) 20
B) 02
C) 25
D) 45

Q178. [Difficult] What is the average number of probes for a successful search in an open addressing hash table with uniform hashing?

A) (1 / alpha) * ln(1 / (1 - alpha))
B) 1 / (1 - alpha)
C) alpha
D) O(n)

Q179. [Easy] Which collision resolution method requires no dynamic memory allocation at runtime?

A) Separate Chaining
B) Open Addressing (Linear Probing, Quadratic Probing, Double Hashing)
C) Binary Search Tree hashing
D) Skip list hashing

Q180. [Easy] If a hash table has size m = 11 and 7 keys are inserted, what is the load factor?

A) 11 / 7
B) 7 / 11 = 0.636
C) 77
D) 4 / 11

Q181. [Easy] What is the folding method of hashing?

A) Dividing key into multiple parts, adding the parts together, and taking modulo m
B) Folding the array into 2D
C) Inverting the bits
D) Multiplying by 2

Q182. [Easy] What is the time complexity to insert a new element in Separate Chaining at the head of the chain?

A) O(1) after computing hash function
B) O(n)
C) O(log n)
D) O(m)

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 10)

QID	Correct Option	Difficulty	Concept Tested
Q164	Option B	Easy	What is the Load Factor (alpha) of a Hash ...
Q165	Option B	Easy	Which collision resolution technique suffe...
Q166	Option A	Medium	What is the probe sequence formula in Doub...
Q167	Option B	Medium	In Open Addressing hashing, what is the ma...
Q168	Option C	Medium	Given hash function h(k) = k mod 7. Keys 1...
Q169	Option C	Easy	What is the worst-case search time complex...
Q170	Option A	Easy	What is the desirable property of a good h...
Q171	Option B	Difficult	Secondary clustering in quadratic probing ...
Q172	Option B	Medium	Why is the table size m usually chosen as ...
Q173	Option B	Easy	In separate chaining, what data structure ...
Q174	Option B	Easy	In Quadratic Probing, the probe sequence i...
Q175	Option B	Medium	What is Rehashing?...
Q176	Option B	Difficult	How is deletion typically handled in an Op...
Q177	Option B	Easy	In the Mid-Square hash method, if key k = ...
Q178	Option A	Difficult	What is the average number of probes for a...
Q179	Option B	Easy	Which collision resolution method requires...
Q180	Option B	Easy	If a hash table has size m = 11 and 7 keys...
Q181	Option A	Easy	What is the folding method of hashing?...
Q182	Option A	Easy	What is the time complexity to insert a ne...

[INFO] Solution for Q164: Correct Option [B]

- Explanation: Load Factor alpha = n / m (Number of keys / Table size). Yeh table ki fullness ko measure karta hai.
- Exam Trap & Shortcut: Do not invert: it is n / m, not m / n.

[INFO] Solution for Q165: Correct Option [B]

- Explanation: Linear probing mein agar kisi index par collision hota hai toh consecutive occupied slots ka cluster ban jata hai (Primary Clustering), jisse search time dramatically badh jata hai.
- Exam Trap & Shortcut: Quadratic probing causes Secondary Clustering; Linear probing causes Primary Clustering.

[INFO] Solution for Q166: Correct Option [A]

- Explanation: Double Hashing probe sequence: `hash(k, i) = (h1(k) + i * h2(k)) % m`. Yahan step size `h2(k)` key par depend karta hai, eliminating clustering.
- Exam Trap & Shortcut: h2(k) must never evaluate to 0 and must be relatively prime to m.

[INFO] Solution for Q167: Correct Option [B]

- Explanation: Open Addressing mein har slot mein maximum 1 key store ho sakti hai. Isliye total keys n kabhi bhi table size m se zyada nahi ho sakti: n <= m, so alpha <= 1.
- Exam Trap & Shortcut: Separate chaining can have alpha > 1, but open addressing CANNOT exceed 1.

[INFO] Solution for Q168: Correct Option [C]

- Explanation: Table size m = 7.
  • Insert 15: 15 % 7 = 1 -> slot 1 is empty -> stored at index 1.
  • Insert 22: 22 % 7 = 1 -> collision at slot 1! Linear probe checks slot 2 -> empty -> stored at index 2.
  • Insert 8: 8 % 7 = 1 -> collision at slot 1! Probe slot 2 -> collision! Probe slot 3 -> empty -> stored at index 3.
  Final index of 8 = 3.
- Exam Trap & Shortcut: Trace each collision probe sequentially: index 1 occupied, index 2 occupied, goes into index 3.

[INFO] Solution for Q169: Correct Option [C]

- Explanation: Worst case tab hota hai jab sabhi n keys same hash slot par collide ho jayein. Saare elements ek single linked list mein chale jaate hain, resulting in O(n) linear search time.
- Exam Trap & Shortcut: Average case is O(1), but WORST case is strictly O(n).

[INFO] Solution for Q170: Correct Option [A]

- Explanation: Uniform hashing distribution minimizes collisions and ensures near O(1) average performance.
- Exam Trap & Shortcut: Uniform distribution and low computation overhead are the primary criteria.

[INFO] Solution for Q171: Correct Option [B]

- Explanation: Agar do alag-alag keys ka initial hash value same ho `h'(k1) == h'(k2)`, toh unka quadratic probe offset sequence `(i, i^2)` identical hota hai. This is called Secondary Clustering.
- Exam Trap & Shortcut: Secondary clustering is milder than primary clustering, but still degrades performance.

[INFO] Solution for Q172: Correct Option [B]

- Explanation: Agar m prime nahi hoga (e.g. power of 2), toh hash values sirf lower-order bits par depend karengi, leading to frequent collisions for structured keys.
- Exam Trap & Shortcut: Prime modulo distributes clustered keys evenly.

[INFO] Solution for Q173: Correct Option [B]

- Explanation: Each bucket in separate chaining maintains a Singly Linked List of colliding keys.
- Exam Trap & Shortcut: Some modern implementations like Java 8 HashMap convert linked list to Red-Black Tree when chain length exceeds 8, but classical definition is Linked List.

[INFO] Solution for Q174: Correct Option [B]

- Explanation: Quadratic probing quadratic polynomial function use karta hai: `(h(k) + c1*i + c2*i^2) % m`. Usually c1 = 1, c2 = 1.
- Exam Trap & Shortcut: Linear uses i; Quadratic uses i^2.

[INFO] Solution for Q175: Correct Option [B]

- Explanation: Jab hash table ka load factor threshold (e.g. 0.75) cross karta hai, performance degrade hone lagti hai. Naya double-sized table banakar sabhi keys ko naye table size par re-insert karne ko Rehashing kehte hain.
- Exam Trap & Shortcut: Rehashing takes O(n) time, but amortized insertion remains O(1).

[INFO] Solution for Q176: Correct Option [B]

- Explanation: Agar slot ko direct empty (NULL) kar denge, toh search algorithm wahan search stop kar dega aur uske aage probe hui keys kabhi nahi milengi. Isliye slot ko tombstone marker se mark kiya jata hai.
- Exam Trap & Shortcut: Directly setting to NULL breaks probe sequences for keys inserted after collisions.

[INFO] Solution for Q177: Correct Option [B]

- Explanation: Mid-Square method mein key ka square calculate karte hain: k = 45 => 45^2 = 2025. 2025 ke 4 digits [2, 0, 2, 5] hain, jisme middle two digits 02 hain. Isliye hash address 02 (ya 2) hoga.
- Exam Trap & Shortcut: Calculate square first: 45 * 45 = 2025, then extract the middle digits 02.

[INFO] Solution for Q178: Correct Option [A]

- Explanation: Standard Knuth formula: Average probes for successful search is `(1 / alpha) * ln(1 / (1 - alpha))`. For unsuccessful search it is `1 / (1 - alpha)`.
- Exam Trap & Shortcut: 1 / (1 - alpha) is for UNSUCCESSFUL search; the ln form is for SUCCESSFUL search.

[INFO] Solution for Q179: Correct Option [B]

- Explanation: Open Addressing ek pre-allocated static array ke andar hi probes dhoondti hai, so no dynamic pointer allocation (malloc) is needed.
- Exam Trap & Shortcut: Separate chaining continuously allocates new nodes via malloc.

[INFO] Solution for Q180: Correct Option [B]

- Explanation: alpha = n / m = 7 / 11 ~= 0.636.
- Exam Trap & Shortcut: alpha = keys / table_size.

[INFO] Solution for Q181: Correct Option [A]

- Explanation: Example: Telephone number 9876543210 -> parts: 987 + 654 + 321 + 0 = sum, then sum % m.
- Exam Trap & Shortcut: Boundary folding folds digits like accordion paper, shift folding adds directly.

[INFO] Solution for Q182: Correct Option [A]

- Explanation: Hash compute karna O(1) hai, aur bucket ke head par node insert karna O(1) hai. Total time = O(1).
- Exam Trap & Shortcut: Inserting at head avoids traversing the existing linked list chain.

Unit 11: Graph Theory & Traversals (Q183 - Q202)

SECTION A: PRACTICE QUESTIONS

Q183. [Easy] According to the Handshaking Lemma, the sum of degrees of all vertices in an undirected graph is equal to:

A) |E|
B) 2 * |E|
C) |V| - 1
D) 2 * |V|

Q184. [Medium] Can a simple undirected graph have 5 vertices, each having degree 3?

A) Yes, always
B) No, because the sum of degrees would be odd (15), which violates the Handshaking Lemma
C) Yes, if it is complete
D) Yes, if it has cycles

Q185. [Easy] How many edges are in a complete undirected graph Kn with n vertices?

A) n
B) n(n - 1)
C) n(n - 1) / 2
D) 2n - 1

Q186. [Medium] An undirected graph has an Eulerian Circuit if and only if it is connected and:

A) Every vertex has an odd degree
B) Every vertex has an even degree
C) Exactly two vertices have odd degree
D) It has no cycles

Q187. [Easy] What is the space complexity of representing a graph with V vertices and E edges using an Adjacency Matrix?

A) O(V + E)
B) O(V^2)
C) O(E^2)
D) O(V log V)

Q188. [Easy] Which traversal algorithm can be used to find the shortest path between two vertices in an unweighted graph?

A) Depth-First Search (DFS)
B) Breadth-First Search (BFS)
C) Inorder Traversal
D) Preorder Traversal

Q189. [Easy] What is the time complexity of BFS or DFS on a graph represented using an Adjacency List?

A) O(V^2)
B) O(V + E)
C) O(V * E)

D) O(log V)

Q190. [Easy] According to Euler's formula for connected planar graphs, the relationship between vertices (V), edges (E), and faces/regions (F) is:

A) V - E + F = 2
B) V + E - F = 2
C) V - E - F = 2
D) V + E + F = 2

Q191. [Easy] In a directed graph, if node 4 has 3 incoming edges and 3 outgoing edges, what are its in-degree, out-degree, and total degree?

A) In-degree = 3, Out-degree = 6, Total = 9
B) In-degree = 3, Out-degree = 3, Total degree = 6
C) In-degree = 6, Out-degree = 3, Total = 9
D) In-degree = 6, Out-degree = 6, Total = 12

Q192. [Medium] What is the chromatic number of a complete graph Kn with n vertices?

A) 1
B) 2
C) n
D) n - 1

Q193. [Easy] A bipartite graph can be colored using how many minimum colors?

A) 1
B) 2
C) 3
D) 4

Q194. [Easy] In an undirected graph with 10 vertices and 15 edges, what is the sum of degrees of all vertices?

A) 15
B) 30
C) 25
D) 45

Q195. [Easy] What is a Hamiltonian Cycle in a graph?

A) A cycle that visits every edge exactly once
B) A closed loop that visits every VERTEX exactly once and returns to starting vertex
C) A tree with no leaves
D) A path of length 2

Q196. [Easy] What is a Spanning Tree of a connected undirected graph G with V vertices?

A) A subgraph that connects all V vertices and has exactly V - 1 edges with no cycles
B) A tree with V edges
C) The complete graph
D) A disconnected subgraph

Q197. [Medium] Which algorithm is used to find the Minimum Spanning Tree (MST) of a weighted graph using a greedy edge-by-edge approach with cycle checking?

A) Kruskal's Algorithm (using Disjoint Set Union)
B) Dijkstra's Algorithm
C) Floyd-Warshall
D) Bellman-Ford

Q198. [Difficult] What is the maximum number of edges in a connected planar simple graph with V >= 3 vertices?

A) 3V - 6
B) 2V - 4
C) V(V-1)/2
D) 3V

Q199. [Medium] What is a Cut Vertex (Articulation Point) in a graph?

A) A vertex with degree 0
B) A vertex whose removal increases the number of connected components (disconnects the graph)
C) A leaf node
D) The root of a tree

Q200. [Easy] Topological Sort can only be performed on which type of graph?

A) Undirected Graphs
B) Directed Acyclic Graphs (DAG)
C) Graphs with negative cycles

D) Complete Graphs Kn

Q201. [Medium] What is the time complexity of Dijkstra's algorithm using a Min-Heap (Priority Queue)?

A) O(V^2)
B) O((V + E) log V)
C) O(V * E)
D) O(V^3)

Q202. [Easy] An undirected graph is called regular if:

A) It has no cycles
B) Every vertex has the exact same degree
C) It is planar
D) It has an even number of vertices

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 11)

QID	Correct Option	Difficulty	Concept Tested
Q183	Option B	Easy	According to the Handshaking Lemma, the su...
Q184	Option B	Medium	Can a simple undirected graph have 5 verti...
Q185	Option C	Easy	How many edges are in a complete undirecte...
Q186	Option B	Medium	An undirected graph has an Eulerian Circui...
Q187	Option B	Easy	What is the space complexity of representi...
Q188	Option B	Easy	Which traversal algorithm can be used to f...
Q189	Option B	Easy	What is the time complexity of BFS or DFS ...
Q190	Option A	Easy	According to Euler's formula for connected...
Q191	Option B	Easy	In a directed graph, if node 4 has 3 incom...
Q192	Option C	Medium	What is the chromatic number of a complete...
Q193	Option B	Easy	A bipartite graph can be colored using how...
Q194	Option B	Easy	In an undirected graph with 10 vertices an...
Q195	Option B	Easy	What is a Hamiltonian Cycle in a graph?...
Q196	Option A	Easy	What is a Spanning Tree of a connected und...
Q197	Option A	Medium	Which algorithm is used to find the Minimu...
Q198	Option A	Difficult	What is the maximum number of edges in a c...
Q199	Option B	Medium	What is a Cut Vertex (Articulation Point) ...
Q200	Option B	Easy	Topological Sort can only be performed on ...
Q201	Option B	Medium	What is the time complexity of Dijkstra's ...
Q202	Option B	Easy	An undirected graph is called regular if:...

[INFO] Solution for Q183: Correct Option [B]

- Explanation: Har edge do vertices par incident hoti hai, isliye har edge total degree mein 2 contribute karti hai: Sum deg(v) = 2 * |E|.
- Exam Trap & Shortcut: Sum is 2 * E, not 2 * V.

[INFO] Solution for Q184: Correct Option [B]

- Explanation: Sum of degrees = 5 * 3 = 15. Handshaking theorem says Sum deg(v) = 2E (must be an even number). 15 is odd, so such a graph is mathematically IMPOSSIBLE!
- Exam Trap & Shortcut: The number of vertices with odd degree MUST always be even!

[INFO] Solution for Q185: Correct Option [C]

- Explanation: Har vertex baaki n - 1 vertices se connect hota hai: n * (n - 1) / 2.
- Exam Trap & Shortcut: For directed graph it is n(n - 1), for undirected it is n(n - 1) / 2.

[INFO] Solution for Q186: Correct Option [B]

- Explanation: Eulerian circuit visits every edge exactly once and returns to the start vertex. Yeh tabhi possible hai jab har vertex ki degree EVEN ho.
- Exam Trap & Shortcut: If exactly 2 vertices have odd degree, it has an Eulerian PATH, not an Eulerian CIRCUIT.

[INFO] Solution for Q187: Correct Option [B]

- Explanation: Adjacency Matrix ek V x V 2D array allocate karti hai, isliye memory hamesha O(V^2) hoti hai, chahe graph mein 0 edges hon ya maximum edges.
- Exam Trap & Shortcut: Adjacency List takes O(V + E), Adjacency Matrix takes O(V^2).

[INFO] Solution for Q188: Correct Option [B]

- Explanation: BFS level-by-level explore karta hai, isliye unweighted graph mein source se kisi bhi target tak pehli baar pahunchne wala path guaranteed shortest path hota hai.
- Exam Trap & Shortcut: DFS goes deep and does NOT find the shortest path in unweighted graphs.

[INFO] Solution for Q189: Correct Option [B]

- Explanation: Har vertex ek baar visit hota hai (O(V)) aur har edge Adjacency List mein traverse hoti hai (O(E)). Total time: O(V + E).
- Exam Trap & Shortcut: If using Adjacency Matrix, the time complexity is O(V^2).

[INFO] Solution for Q190: Correct Option [A]

- Explanation: Euler's Planar Graph Formula: V - E + F = 2.
- Exam Trap & Shortcut: Remember the signs: V (positive), E (negative), F (positive): V - E + F = 2.

[INFO] Solution for Q191: Correct Option [B]

- Explanation: In-degree = incoming edges = 3. Out-degree = outgoing edges = 3. Total degree = In-degree + Out-degree = 6. (Note: Original course slide had a typo stating outdegree 6).
- Exam Trap & Shortcut: Do not confuse total degree (6) with outdegree (3).

[INFO] Solution for Q192: Correct Option [C]

- Explanation: Kn mein har vertex har doosre vertex se adjacent hota hai. Isliye kisi bhi do vertices ko same color nahi mil sakta. Hence, chromatic number = n colors.
- Exam Trap & Shortcut: Every vertex in Kn requires a distinct unique color.

[INFO] Solution for Q193: Correct Option [B]

- Explanation: Bipartite graph ke vertices do disjoint sets mein divide hote hain aur edges sirf ek set se doosre set ke beech hoti hain. Set 1 ko color A aur Set 2 ko color B de sakte hain. Chromatic number = 2.
- Exam Trap & Shortcut: A graph is bipartite if and only if it has no odd-length cycles and can be 2-colored.

[INFO] Solution for Q194: Correct Option [B]

- Explanation: By Handshaking Lemma: Sum deg(v) = 2 * |E| = 2 * 15 = 30.
- Exam Trap & Shortcut: Number of vertices (10) is irrelevant here; sum is strictly 2 * E.

[INFO] Solution for Q195: Correct Option [B]

- Explanation: Eulerian visits every EDGE once; Hamiltonian visits every VERTEX once.
- Exam Trap & Shortcut: Eulerian = Edges; Hamiltonian = Vertices.

[INFO] Solution for Q196: Correct Option [A]

- Explanation: Spanning tree graph ke sabhi V vertices ko bina kisi cycle ke connect karta hai, containing exactly V - 1 edges.
- Exam Trap & Shortcut: Number of edges in any spanning tree of V vertices is strictly V - 1.

[INFO] Solution for Q197: Correct Option [A]

- Explanation: Kruskal's algorithm edges ko weight ke ascending order mein sort karta hai aur cycle avoid karte hue MST banata hai (using Union-Find). Time: O(E log E).
- Exam Trap & Shortcut: Dijkstra finds Single Source Shortest Path, not MST.

[INFO] Solution for Q198: Correct Option [A]

- Explanation: Planar graph theorem: Har face kam se kam 3 edges se bound hota hai (2E >= 3F). Euler formula V - E + F = 2 mein substitute karne par: E <= 3V - 6.
- Exam Trap & Shortcut: If the planar graph is bipartite (no triangles, 2E >= 4F), E <= 2V - 4.

[INFO] Solution for Q199: Correct Option [B]

- Explanation: Articulation point woh critical vertex hota hai jisko remove karne par graph disconnected ho jata hai. Discovered via DFS in O(V + E).
- Exam Trap & Shortcut: Bridge is the equivalent concept for an EDGE whose removal disconnects the graph.

[INFO] Solution for Q200: Correct Option [B]

- Explanation: Topological sort linear ordering deta hai jisme har directed edge u -> v ke liye u hamesha v se pehle aata hai. Yeh sirf DAGs par possible hai.
- Exam Trap & Shortcut: If the graph has a directed cycle, topological sort is impossible.

[INFO] Solution for Q201: Correct Option [B]

- Explanation: Min-Heap ke saath Dijkstra Single Source Shortest Path find karta hai in O((V + E) log V) time.
- Exam Trap & Shortcut: With simple array it is O(V^2); with binary heap it is O((V + E) log V).

[INFO] Solution for Q202: Correct Option [B]

- Explanation: k-regular graph mein har vertex ki degree k hoti hai (e.g. Petersen graph is 3-regular).
- Exam Trap & Shortcut: Complete graph Kn is (n-1)-regular.

Unit 12: Searching & Sorting Algorithms (Q203 - Q220)

SECTION A: PRACTICE QUESTIONS

Q203. [Medium] Which sorting algorithm makes the minimum number of memory swaps (exactly O(n)) in the worst case?

A) Bubble Sort
B) Selection Sort
C) Insertion Sort
D) Quick Sort

Q204. [Easy] What is the worst-case time complexity of Quick Sort, and under what condition does it occur?

A) O(n log n), when array is randomly shuffled
B) O(n^2), when array is already sorted and first/last element is picked as pivot
C) O(n), when all elements are distinct
D) O(n log n), when array has duplicates

Q205. [Easy] Which of the following sorting algorithms is STABLE and has a worst-case time complexity of O(n log n)?

A) Quick Sort
B) Heap Sort
C) Merge Sort
D) Selection Sort

Q206. [Easy] What is the best-case time complexity of Insertion Sort, and when does it occur?

A) O(n^2), when array is reverse sorted
B) O(n), when array is already sorted
C) O(n log n), when array has random order
D) O(1), when array has 10 elements

Q207. [Easy] How much auxiliary (extra) memory does Merge Sort require to sort an array of size n?

A) O(1)
B) O(log n)
C) O(n)
D) O(n^2)

Q208. [Medium] What is the time complexity to build a Binary Max-Heap from an unsorted array of n elements (Build-Heap operation)?

A) O(n log n)
B) O(n)
C) O(n^2)
D) O(log n)

Q209. [Easy] Which sorting algorithm is UNSTABLE among the following?

A) Bubble Sort
B) Insertion Sort
C) Selection Sort
D) Merge Sort

Q210. [Medium] What is the maximum number of comparisons required to search for an element in a sorted array of 1024 elements using Binary Search?

A) 10
B) 11
C) 1024
D) 512

Q211. [Easy] Which sorting algorithm has the best average-case time complexity of O(n + k), where k is the range of input values?

A) Counting Sort
B) Quick Sort
C) Heap Sort
D) Bubble Sort

Q212. [Medium] In Max-Heap, where is the second largest element always located?

A) At the root
B) At index 1 or index 2 (children of root)
C) At any leaf node
D) At the bottom right

Q213. [Easy] What is an In-Place sorting algorithm?

A) An algorithm that requires no CPU registers
B) An algorithm that requires O(1) or O(log n) auxiliary space beyond the input array
C) An algorithm that sorts in O(n) time
D) An algorithm that does not modify the original array

Q214. [Easy] Radix Sort uses which underlying stable sorting algorithm to sort keys digit by digit?

A) Counting Sort
B) Quick Sort
C) Heap Sort
D) Selection Sort

Q215. [Medium] If an array is already sorted, which algorithm will perform the MAXIMUM number of comparisons?

A) Insertion Sort
B) Selection Sort
C) Bubble Sort with flag
D) Binary Search

Q216. [Easy] What is the recurrence relation for the best-case of Quick Sort with equal partitioning?

A) T(n) = 2T(n/2) + O(n)
B) T(n) = T(n - 1) + O(n)
C) T(n) = 2T(n/2) + O(1)
D) T(n) = T(n/2) + O(1)

Q217. [Easy] What is the time complexity of Heapify operation on a binary heap of size n?

A) O(1)
B) O(log n)
C) O(n)
D) O(n log n)

Q218. [Medium] What is the lower bound on the number of comparisons for ANY comparison-based sorting algorithm in the worst case?

A) Omega(n)
B) Omega(n log n)
C) Omega(n^2)
D) Omega(log n)

Q219. [Easy] In an array representation of a binary heap with 0-based indexing, where is the right child of element at index i?

A) 2 * i
B) 2 * i + 1
C) 2 * i + 2
D) floor((i - 1) / 2)

Q220. [Medium] Which algorithm is best suited for sorting a linked list efficiently without random access?

A) Quick Sort
B) Merge Sort
C) Heap Sort
D) Binary Search

SECTION B: DETAILED HINGLISH SOLUTIONS & EXAM TRAPS (Unit 12)

QID	Correct Option	Difficulty	Concept Tested
Q203	Option B	Medium	Which sorting algorithm makes the minimum ...
Q204	Option B	Easy	What is the worst-case time complexity of ...
Q205	Option C	Easy	Which of the following sorting algorithms ...
Q206	Option B	Easy	What is the best-case time complexity of I...
Q207	Option C	Easy	How much auxiliary (extra) memory does Mer...
Q208	Option B	Medium	What is the time complexity to build a Bin...
Q209	Option C	Easy	Which sorting algorithm is UNSTABLE among ...
Q210	Option B	Medium	What is the maximum number of comparisons ...
Q211	Option A	Easy	Which sorting algorithm has the best avera...
Q212	Option B	Medium	In Max-Heap, where is the second largest e...
Q213	Option B	Easy	What is an In-Place sorting algorithm?...
Q214	Option A	Easy	Radix Sort uses which underlying stable so...
Q215	Option B	Medium	If an array is already sorted, which algor...
Q216	Option A	Easy	What is the recurrence relation for the be...
Q217	Option B	Easy	What is the time complexity of Heapify ope...
Q218	Option B	Medium	What is the lower bound on the number of c...
Q219	Option C	Easy	In an array representation of a binary hea...
Q220	Option B	Medium	Which algorithm is best suited for sorting...

[INFO] Solution for Q203: Correct Option [B]

- Explanation: Selection sort har pass mein exactly ek swap karta hai minimum element ko uske position par laane ke liye. Total swaps = n - 1 = O(n). Isliye jab memory writes expensive hon (jaise EEPROM/Flash), Selection Sort prefer hota hai.
- Exam Trap & Shortcut: Comparisons are O(n^2), but SWAPS are strictly O(n)!

[INFO] Solution for Q204: Correct Option [B]

- Explanation: Jab array already sorted ya reverse-sorted ho aur pehla ya aakhiri element pivot chuna jaye, toh partition completely unbalanced (0 aur n-1) hota hai. Recurrence: T(n) = T(n-1) + O(n) = O(n^2).
- Exam Trap & Shortcut: Quick sort is NOT O(n log n) in all cases; its worst case is O(n^2).

[INFO] Solution for Q205: Correct Option [C]

- Explanation: Merge Sort guarantees O(n log n) in best, average, and worst cases, and it is STABLE (preserves relative order of equal keys). Heap sort and Quick sort are unstable.
- Exam Trap & Shortcut: Heap sort is O(n log n) worst case, but it is UNSTABLE.

[INFO] Solution for Q206: Correct Option [B]

- Explanation: Jab array already sorted hota hai, Insertion Sort har element ko sirf ek comparison ke baad aage badha deta hai (0 inner loop shifts). Total comparisons = n - 1 = O(n).
- Exam Trap & Shortcut: Insertion sort is an adaptive sorting algorithm.

[INFO] Solution for Q207: Correct Option [C]

- Explanation: Merge sort ko arrays merge karne ke liye ek temporary array allocate karni padti hai jiska size n hota hai. Auxiliary space = O(n). This makes it an OUT-OF-PLACE sort for arrays.
- Exam Trap & Shortcut: Merge sort is in-place for Linked Lists, but out-of-place (O(n) space) for Arrays.

[INFO] Solution for Q208: Correct Option [B]

- Explanation: Mathematical analysis of bottom-up heap construction shows that sum of heights over all nodes converges: Sum (n / 2^(h+1)) * O(h) = O(n). Building a heap takes linear O(n) time, NOT O(n log n).
- Exam Trap & Shortcut: Inserting elements one by one takes O(n log n), but Build-Heap bottom-up takes strictly O(n)!

[INFO] Solution for Q209: Correct Option [C]

- Explanation: Selection sort long-distance swaps karta hai jo equal keys ke relative order ko change kar deta hai (e.g. sorting [4a, 4b, 2] -> 2 swaps with 4a -> [2, 4b, 4a]).
- Exam Trap & Shortcut: Remember the unstable trio: Quick Sort, Heap Sort, Selection Sort.

[INFO] Solution for Q210: Correct Option [B]

- Explanation: Max comparisons = floor(log2(n)) + 1. For n = 1024: log2(1024) = 10. Maximum comparisons = 10 + 1 = 11 comparisons (in worst case when element is not found or at the leaf).
- Exam Trap & Shortcut: log2(1024) is 10, but in worst case (not found) it performs 11 comparisons.

[INFO] Solution for Q211: Correct Option [A]

- Explanation: Counting sort non-comparison based algorithm hai jo element frequencies count karke O(n + k) time mein sort karta hai.
- Exam Trap & Shortcut: It is efficient only when range k is not significantly larger than n.

[INFO] Solution for Q212: Correct Option [B]

- Explanation: Root contains the largest element. Second largest element can only be one of the immediate children of the root (Level 1, index 1 or 2).
- Exam Trap & Shortcut: It cannot be at level 2 because root's children are larger than their respective subtrees.

[INFO] Solution for Q213: Correct Option [B]

- Explanation: In-place sorting algorithm input array ke andar hi elements ko rearrange karta hai and requires negligible extra space (O(1) auxiliary memory).
- Exam Trap & Shortcut: Merge sort is out-of-place because it needs O(n) auxiliary space.

[INFO] Solution for Q214: Correct Option [A]

- Explanation: Radix sort least significant digit (LSD) se most significant digit tak sort karta hai. Digit sorting ke liye Counting Sort use hota hai kyunki digits 0..9 limited range ki hoti hain aur Counting Sort stable hai.
- Exam Trap & Shortcut: Radix sort requires a STABLE subroutine; unstable sort will destroy previous digit ordering.

[INFO] Solution for Q215: Correct Option [B]

- Explanation: Selection sort input order check nahi karta; woh hamesha poore unsorted part ko scan karta hai: exactly n(n-1)/2 comparisons in ALL cases! Insertion sort and Bubble sort with flag will only make n-1 comparisons.
- Exam Trap & Shortcut: Selection sort is NOT adaptive: it works equally hard on sorted data!

[INFO] Solution for Q216: Correct Option [A]

- Explanation: Best case partition array ko do equal halves (n/2 aur n/2) mein baant ta hai with O(n) partition work: T(n) = 2T(n/2) + O(n) = O(n log n).
- Exam Trap & Shortcut: Worst case recurrence is T(n) = T(n-1) + O(n).

[INFO] Solution for Q217: Correct Option [B]

- Explanation: Heapify node ko downwards push karta hai till leaf. Maximum path length tree height hoti hai: O(log n).
- Exam Trap & Shortcut: Build-Heap is O(n), but single Heapify is O(log n).

[INFO] Solution for Q218: Correct Option [B]

- Explanation: Decision tree model theorem: An n-element array has n! permutations. Decision tree height must be at least log2(n!) = Omega(n log n). No comparison-based sort can beat Omega(n log n) worst case.
- Exam Trap & Shortcut: Non-comparison sorts (Counting, Radix) can achieve O(n) because they do not use comparisons.

[INFO] Solution for Q219: Correct Option [C]

- Explanation: In 0-based indexing: Left child = 2i + 1, Right child = 2i + 2, Parent = floor((i - 1) / 2).
- Exam Trap & Shortcut: In 1-based indexing, right child is 2i + 1. In 0-based indexing, it is 2i + 2.

[INFO] Solution for Q220: Correct Option [B]

- Explanation: Merge sort does not require random access (it splits list using slow/fast pointers in O(n) and merges sequentially in O(n)), and can be done IN-PLACE on linked lists without extra array allocation. Ideal for Linked Lists!
- Exam Trap & Shortcut: Quick sort and Heap sort require random indexing and are inefficient for linked lists.